

Módulo Colores en tienda — LhexIA

Ferretería SD

Fecha: 2026-05-30 · Repo: sistema_ventas_limpio

1. Qué queremos (objetivo de producto)

- Asistente en mostrador / tablet / TV para que el cliente elija ambiente, color y cantidad de pintura.
- Mostrar solo productos con stock real en tienda (ERP), no tintometría ni mezcla — eso es fase 2.
- Vista previa del color en un ambiente (comedor, cocina, dormitorio, baño, living, fachada).
- Cotización referencial (galones, complementos, total) y acciones: carrito / pedir en caja / WhatsApp.
- Módulo aislado del catálogo vitrina público: se habilita desde caja con token o lab con preview flag.
- Integración con Liz (asistente flotante), no barra intrusiva en el wizard.
- Calidad visual comparable a Sodimac Fábrica de Color (referencia, no copia de marca).

2. Problemas encontrados y estado

Problema	Causa	Estado
404 al abrir lab	Faltaba VITRINA_FABRICA_COLOR_PREVIEW=1 y reinicio Flask.	Resuelto
UI poco premium vs Sodimac	Grid de fotos grandes, workspace split, panel lateral.	Mejorado; iteraciones pendientes
Fotos no correspondían al texto	URLs Pexels/Unsplash incorrectas o cache.	Corregido con fotos curadas por ambiente
Color teñía toda la foto	Overlay CSS mix-blend-mode sobre rectángulo completo.	Reemplazado por canvas + máscaras PNG
Tinte verde global	Blur gaussiano en máscara dejaba alpha > 0 en muebles.	Corregido: máscaras binarias sin blur
Ácido muriático en paleta amarilla	Filtro es_pintura_con_color incompleto.	Exclusiones ampliadas en pintura_stock_palette_service
Barra rosa Liz confusa	Duplicaba al botón flotante Liz.	Eliminada del wizard
Visualizador vs	Sodimac usa máscaras trazadas a mano	Gap conocido — ver sección

Sodimac	/ SDK por foto.	5
Etiqueta Dormitorio con foto Living	dormitorio.jpg era foto de living.	Corregido

3. Arquitectura y rutas

3.1 Activación

- .env.local: VITRINA_FABRICA_COLOR_PREVIEW=1 (lab)
- Caja: POST /api/caja/modulo-pinturas/habilitar → token cliente
- Lab: GET /modulos/pinturas/lab → GET /modulos/pinturas/lab/iniciar
- Cliente: GET /modulos/pinturas/<token>

3.2 API

- POST /api/modulos/pinturas/<token>/cotizar
- POST /api/modulos/pinturas/<token>/liz-tip

3.3 Archivos principales

- blueprints/modulo_pinturas.py
- services/fabrica_color_service.py
- services/pintura_stock_palette_service.py
- services/modulo_pinturas_session_service.py
- services/pintura_cartilla_service.py
- templates/modulos/fabrica_color.html
- templates/modulos/pinturas_lab.html
- templates/modulos/includes/header_modulo_pinturas.html
- static/js/fabrica-color.js
- static/css/fabrica-color.css
- scripts/generar_mascaras_ambientes.py
- tests/test_modulo_pinturas.py

3.4 Assets

- static/img/fabrica-color/ambientes/*.jpg — fotos por ambiente
- static/img/fabrica-color/masks/*.png — máscaras de muro (generar con scripts/generar_mascaras_ambientes.py)
- data/pintura_cartilla_sd.json — cartilla fase 2 (FABRICA_COLOR_FUENTE=cartilla)

3.5 Flujo wizard (4 pasos)

- Paso 1: grid grande de ambientes → auto avanza al paso 2
- Paso 2: paleta stock ERP + brillo + preview canvas
- Paso 3: m² → galones estimados
- Paso 4: resumen, complementos, pedir en caja / carrito

4. Variables de entorno

- VITRINA_FABRICA_COLOR_PREVIEW=1 — habilita lab
- FABRICA_COLOR_FUENTE=stock_erp (default) | cartilla (fase 2 tintometría)

5. Gap vs Sodimac y próximos pasos

Sodimac usa fotos propias + máscaras de muro por píxel (SDK proveedor pintura). Nosotros usamos stock genérico + polígonos/máscaras aproximadas. El color solo pinta bien la franja superior del muro; no segmenta detrás del sofá como Sodimac.

Para empatar calidad visual (recomendado post SD-1 piso):

- 6 fotos reales o renders alineados a Santo Domingo
- 6 máscaras PNG trazadas a mano (Photoshop/Figma) por foto
- Regenerar: python scripts/generar_mascaras_ambientes.py
- Opcional fase 2: FABRICA_COLOR_FUENTE=cartilla + tintometría Kölor/Topex

6. Comandos útiles

- pytest tests/test_modulo_pinturas.py -v
- python scripts/generar_mascaras_ambientes.py
- python scripts/generar_doc_modulo_pinturas.py

Anexo A — Código fuente completo

Snapshot al generar este documento. Archivos en orden de dependencia.

blueprints/modulo_pinturas.py

```
"""Módulo cliente Fábrica de Color – fuera del catálogo vitrina pública."""
from __future__ import annotations

import os

from flask import abort, jsonify, render_template, request, url_for
from flask_login import current_user

from services import fabrica_color_service as fc
from services import modulo_pinturas_session_service as ms
from services import vitrina_tienda_service as vt

def _empresa_ctx():
    from blueprints._app_ref import app_module

    m = app_module()
    cfg = m.obtener_config_empresa()
    nombre = (cfg.get('nombre_comercial') or cfg.get('razon_social') or vt.TIENDA_TITULO_DEFAULT).strip()
    return cfg, nombre

def _wa_base():
    dig = ''.join(c for c in (os.getenv('WHATSAPP_VENTAS', '') or '') if c.isdigit())
    return f'https://wa.me/{dig}' if dig else None

def _carrito_template_ctx(slug: str, nombre_tienda: str):
    dig = ''.join(c for c in (os.getenv('WHATSAPP_VENTAS', '') or '') if c.isdigit())
    return {
        'carrito_config': {
            'storage_key': vt.CARRITO_STORAGE_KEY,
            'max_qty': vt.CARRITO_MAX_CANTIDAD,
            'whatsapp': dig,
            'nombre_tienda': nombre_tienda,
            'whatsapp_api_url': url_for('tienda_api_carrito_whatsapp', slug=slug),
            'vale_api_url': url_for('tienda_api_carrito_vale', slug=slug),
        },
    }

def _fabrica_assistant_ctx(slug: str):
    from blueprints.tienda_publica import _assistant_template_ctx

    ctx = _assistant_template_ctx(slug)
    ctx['assistant_chips'] = [
        '¿Qué brillo para dormitorio?',
        '¿Cuánto rinde un galón?',
        'Complementos para pintar',
        '¿Pintura exterior para fachada?',
    ]
    return ctx

def _canonical(path: str) -> str:
    from blueprints._app_ref import app_module

    m = app_module()
    try:
        return m._absolute_public_url(path)
    except Exception:
        return path
```

```

def _render_wizard(token: str, sesion: dict):
    if not vt.tienda_habilitada():
        abort(404)
    slug = vt.TIENDA_SLUG_SD
    cfg, nombre_tienda = _empresa_ctx()
    base_path = f'/modulos/pinturas/{token}'
    tv_mode = (request.args.get('tv') or '').strip().lower() in ('1', 'true', 'si', 'yes')
    if sesion.get('modo') == 'caja':
        tv_mode = True
    initial = fc.payload_inicial()
    scene_urls = {
        key: url_for('static', filename=path)
        for key, path in initial.get('scene_assets', {}).items()
    }
    for amb in initial.get('ambientes') or []:
        photo = amb.get('photo')
        if photo:
            amb['photo_url'] = url_for('static', filename=photo) + '?v=mask-v3-20260530'
        mask = amb.get('mask')
        if mask:
            amb['mask_url'] = url_for('static', filename=mask) + '?v=mask-v3-20260530'
    return render_template(
        'modulos/fabrica_color.html',
        slug=slug,
        nombre_tienda=nombre_tienda,
        empresa_cfg=cfg,
        wa_base=_wa_base(),
        sesion_modulo=sesion,
        es_preview_lab=sesion.get('modo') == 'lab',
        tv_mode=tv_mode,
        fabrica_config={
            'initial': initial,
            'cotizar_url': url_for('modulo_pinturas_api_cotizar', token=token),
            'liz_tip_url': url_for('modulo_pinturas_api_liz_tip', token=token),
            'modulo_home_url': url_for('modulo_pinturas_lab') if sesion.get('modo') == 'lab' else None,
            'modo_caja': sesion.get('modo') == 'caja',
            'habilitado_por': (sesion.get('nombre') or '').strip(),
            'tv_mode': tv_mode,
            'scene_urls': scene_urls,
        },
        **fabrica_assistant_ctx(slug),
        page_canonical=_canonical(base_path),
        page_meta_description=(
            f'Colores en tienda {nombre_tienda}: ambiente, producto con stock, cantidad y cotización.'
        ),
    )

def modulo_pinturas_lab():
    if not ms.preview_habilitado():
        abort(404)
    cfg, nombre_tienda = _empresa_ctx()
    return render_template(
        'modulos/pinturas_lab.html',
        nombre_tienda=nombre_tienda,
        empresa_cfg=cfg,
        wizard_url=url_for('modulo_pinturas_lab_iniciar'),
        vitrina_url=url_for('tienda_vitrina', slug=vt.TIENDA_SLUG_SD),
    )

def modulo_pinturas_lab_iniciar():
    if not ms.preview_habilitado():
        abort(404)
    sesion = ms.validar_acceso(ms.LAB_TOKEN)
    if not sesion:

```

```

        abort(404)
    return _render_wizard(ms.LAB_TOKEN, sesion)

def modulo_pinturas_wizard(token: str):
    if token == ms.LAB_TOKEN:
        abort(404)
    sesion = ms.validar_acceso(token)
    if not sesion:
        abort(404)
    return _render_wizard(token, sesion)

def modulo_pinturas_api_cotizar(token: str):
    if not ms.validar_acceso(token):
        return jsonify({'ok': False, 'error': 'not_found'}), 404
    data = request.get_json(silent=True) or {}
    try:
        m2 = float(data.get('m2') or 0)
    except (TypeError, ValueError):
        m2 = 0
    producto_id = data.get('producto_id')
    try:
        pid = int(producto_id) if producto_id is not None else None
    except (TypeError, ValueError):
        pid = None
    try:
        out = fc.cotizar_proyecto(
            ambiente_id=(data.get('ambiente_id') or '').strip(),
            color_id=(data.get('color_id') or '').strip(),
            brillo_id=(data.get('brillo_id') or '').strip(),
            m2=m2,
            calidad_id=(data.get('calidad_id') or 'standard').strip(),
            producto_id=pid,
        )
        return jsonify(out)
    except Exception as ex:
        import logging

        logging.getLogger(__name__).exception('modulo_pinturas_api_cotizar: %s', ex)
        return jsonify({'ok': False, 'error': 'cotizar_error', 'mensaje': str(ex)[:200]}), 500

def modulo_pinturas_api_liz_tip(token: str):
    if not ms.validar_acceso(token):
        return jsonify({'ok': False, 'error': 'not_found'}), 404
    data = request.get_json(silent=True) or {}
    try:
        step = int(data.get('step') or 1)
    except (TypeError, ValueError):
        step = 1
    try:
        m2 = float(data.get('m2') or 0)
    except (TypeError, ValueError):
        m2 = 12.0
    try:
        out = fc.liz_tip_wizard(
            step=step,
            ambiente_id=(data.get('ambiente_id') or '').strip(),
            color_id=(data.get('color_id') or '').strip(),
            brillo_id=(data.get('brillo_id') or '').strip(),
            m2=m2,
        )
        return jsonify(out)
    except Exception as ex:
        import logging

        logging.getLogger(__name__).exception('modulo_pinturas_api_liz_tip: %s', ex)

```

```

        return jsonify({'ok': True, 'tip': '', 'fuente': 'reglas'})

def api_caja_habilitar_modulo_pinturas():
    from blueprints._app_ref import app_module

    m = app_module()
    if not current_user.is_authenticated:
        return jsonify({'ok': False, 'error': 'auth_required'}), 401
    if not (
        m.usuario_tiene_permiso('caja_cobrar_vale')
        or m.usuario_tiene_permiso('pos_emitir_vale')
        or m.usuario_tiene_permiso('gestionar_usuarios')
    ):
        return jsonify({'ok': False, 'error': 'forbidden'}), 403
    try:
        out = ms.crear_sesion_modulo_pinturas(
            usuario_id=int(current_user.id),
            usuario_nombre=(getattr(current_user, 'nombre', None) or getattr(current_user, 'username', '')) or
'Mostrador'),
        )
        try:
            out['url_absoluta'] = m._absolute_public_url(out['path'])
        except Exception:
            out['url_absoluta'] = out['path']
        return jsonify(out)
    except Exception as ex:
        import logging

        logging.getLogger(__name__).exception('api_caja_habilitar_modulo_pinturas: %s', ex)
        return jsonify({'ok': False, 'error': 'session_error', 'mensaje': str(ex)[:200]}), 500

def register_modulo_pinturas_routes(app) -> None:
    app.add_url_rule(
        '/modulos/pinturas/lab',
        view_func=modulo_pinturas_lab,
        methods=['GET'],
        endpoint='modulo_pinturas_lab',
    )
    app.add_url_rule(
        '/modulos/pinturas/lab/iniciar',
        view_func=modulo_pinturas_lab_iniciar,
        methods=['GET'],
        endpoint='modulo_pinturas_lab_iniciar',
    )
    app.add_url_rule(
        '/modulos/pinturas/<token>',
        view_func=modulo_pinturas_wizard,
        methods=['GET'],
        endpoint='modulo_pinturas_wizard',
    )
    app.add_url_rule(
        '/api/modulos/pinturas/<token>/cotizar',
        view_func=modulo_pinturas_api_cotizar,
        methods=['POST'],
        endpoint='modulo_pinturas_api_cotizar',
    )
    app.add_url_rule(
        '/api/modulos/pinturas/<token>/liz-tip',
        view_func=modulo_pinturas_api_liz_tip,
        methods=['POST'],
        endpoint='modulo_pinturas_api_liz_tip',
    )
    app.add_url_rule(
        '/api/caja/modulo-pinturas/habilitar',
        view_func=api_caja_habilitar_modulo_pinturas,
        methods=['POST'],
    )

```

```

        endpoint='api_caja_modulo_pinturas_habilitar',
    )

# Rutas legacy vitrina → 404 (módulo ya no vive en catálogo)
def _legacy_fabrica_404(*_a, **_k):
    abort(404)

app.add_url_rule(
    '/tienda/<slug>/fabrica-de-color',
    view_func=_legacy_fabrica_404,
    methods=['GET'],
    endpoint='tienda_fabrica_color_legacy',
)
app.add_url_rule(
    '/api/tienda/<slug>/fabrica-color/cotizar',
    view_func=_legacy_fabrica_404,
    methods=['POST'],
    endpoint='tienda_api_fabrica_cotizar_legacy',
)

```

services/fabrica_color_service.py

```

"""
Fábrica de Color LhexIA – wizard pinturas (módulo cliente Santo Domingo).
Cartilla Kölor/Topex + productos ERP rubro Pinturas + complementos producto_relacion.
"""
from __future__ import annotations

import math
import os
import re
from typing import Any

from services import pintura_cartilla_service as cartilla
from services import pintura_stock_palette_service as stock_palette

RENDIMIENTO_M2_GALON = 35.0
MANOS_DEFAULT = 2
LITROS_POR_GALON = 3.785

_SCENE_BG = {
    'interior': 'img/fabrica-color/scene-living.svg',
    'bano': 'img/fabrica-color/scene-bano.svg',
    'cocina': 'img/fabrica-color/scene-cocina.svg',
    'fachada': 'img/fabrica-color/scene-fachada.svg',
}

def _ambiente_photo(ambiente_id: str) -> str:
    return f'img/fabrica-color/ambientes/{ambiente_id}.jpg'

def _ambiente_mask(ambiente_id: str) -> str:
    return f'img/fabrica-color/masks/{ambiente_id}.png'

AMBIENTES: list[dict[str, Any]] = [
    {
        "id": "comedor",
        "nombre": "Comedor",
        "icono": "fa-utensils",
        "uso": "interior",
        "scene": "interior",
        "scene_bg": _SCENE_BG['interior'],
    }

```



```

    "photo": _ambiente_photo('comedor'),
    "mask": _ambiente_mask('comedor'),
    "grid_class": "fc-amb-card--hero",
    "wall_polygons": [
        [[0.0, 0.0], [1.0, 0.0], [1.0, 0.48], [0.0, 0.48]],
    ],
    "wall_exclusions": [
        [0.50, 0.78, 0.42, 0.18],
        [0.15, 0.82, 0.12, 0.10],
    ],
    "tip": "Ideal para espacios de reunión con buena luz natural.",
},
{
    "id": "cocina",
    "nombre": "Cocina",
    "icono": "fa-kitchen-set",
    "uso": "interior",
    "scene": "cocina",
    "scene_bg": _SCENE_BG['cocina'],
    "photo": _ambiente_photo('cocina'),
    "mask": _ambiente_mask('cocina'),
    "wall_polygons": [
        [[0.0, 0.0], [1.0, 0.0], [1.0, 0.40], [0.0, 0.40]],
    ],
    "wall_exclusions": [
        [0.70, 0.65, 0.22, 0.30],
        [0.30, 0.72, 0.25, 0.20],
    ],
    "tip": "Satinado facilita limpieza de salpicaduras.",
},
{
    "id": "dormitorio",
    "nombre": "Dormitorio",
    "icono": "fa-bed",
    "uso": "interior",
    "scene": "interior",
    "scene_bg": _SCENE_BG['interior'],
    "photo": _ambiente_photo('dormitorio'),
    "mask": _ambiente_mask('dormitorio'),
    "wall_polygons": [
        [[0.0, 0.0], [1.0, 0.0], [1.0, 0.42], [0.0, 0.42]],
    ],
    "wall_exclusions": [
        [0.50, 0.62, 0.48, 0.28],
        [0.12, 0.35, 0.10, 0.12],
    ],
    "tip": "Prefiera tonos suaves y acabado mate para descanso.",
},
{
    "id": "bano",
    "nombre": "Baño",
    "icono": "fa-bath",
    "uso": "interior",
    "scene": "bano",
    "scene_bg": _SCENE_BG['bano'],
    "photo": _ambiente_photo('bano'),
    "mask": _ambiente_mask('bano'),
    "wall_polygons": [
        [[0.0, 0.0], [1.0, 0.0], [1.0, 0.45], [0.0, 0.45]],
    ],
    "wall_exclusions": [
        [0.48, 0.70, 0.30, 0.22],
        [0.80, 0.55, 0.14, 0.20],
    ],
    "tip": "Use pintura lavable o satinada por humedad.",
},
{
    "id": "living",

```

```

        "nombre": "Living",
        "icono": "fa-couch",
        "uso": "interior",
        "scene": "interior",
        "scene_bg": _SCENE_BG['interior'],
        "photo": _ambiente_photo('living'),
        "mask": _ambiente_mask('living'),
        "wall_polygons": [
            [[0.0, 0.0], [1.0, 0.0], [1.0, 0.30], [0.0, 0.30]],
        ],
        "wall_exclusions": [
            [0.12, 0.10, 0.07, 0.06],
            [0.50, 0.08, 0.09, 0.06],
            [0.86, 0.12, 0.07, 0.07],
        ],
        "tip": "Combine color de acento en muro principal.",
    },
    {
        "id": "fachada",
        "nombre": "Fachada",
        "icono": "fa-house-chimney",
        "uso": "exterior",
        "scene": "fachada",
        "scene_bg": _SCENE_BG['fachada'],
        "photo": _ambiente_photo('fachada'),
        "mask": _ambiente_mask('fachada'),
        "wall_polygons": [
            [[0.12, 0.22], [0.88, 0.22], [0.88, 0.76], [0.12, 0.76]],
        ],
        "wall_exclusions": [
            [0.50, 0.52, 0.08, 0.14],
            [0.28, 0.58, 0.06, 0.10],
            [0.72, 0.58, 0.06, 0.10],
        ],
        "tip": "Elija pintura exterior con protección UV.",
    },
]

BRILLOS: list[dict[str, Any]] = [
    {
        "id": "mate",
        "nombre": "Mate",
        "desc": "Sin reflejo · disimula imperfecciones",
        "ideal": "Paredes y cielos · dormitorios y living",
    },
    {
        "id": "satinado",
        "nombre": "Satinado",
        "desc": "Brillo suave · fácil de limpiar",
        "ideal": "Cocina, baño y zonas de paso",
    },
    {
        "id": "semi_brillo",
        "nombre": "Semi brillo",
        "desc": "Mayor luminosidad · resistente",
        "ideal": "Detalles, muebles y madera pintada",
    },
]

CALIDADES: list[dict[str, Any]] = [
    {"id": "economica", "nombre": "Económica", "badge": "Mejor precio"},
    {"id": "standard", "nombre": "Estándar", "badge": "Más elegida"},
    {"id": "premium", "nombre": "Premium", "badge": "Máxima cobertura"},
]

_FAMILIAS_INTERIOR = ('blanco', 'beige', 'amarillo', 'verde', 'azul', 'gris', 'rojo', 'neutro')

```

```

def _fuente_colores() -> str:
    """stock_erp = solo productos con stock tienda (fase 1). cartilla = tintometría (fase 2)."""
    return (os.getenv('FABRICA_COLOR_FUENTE', 'stock_erp') or 'stock_erp').strip().lower()

def _modo_stock() -> bool:
    return _fuente_colores() in ('stock', 'stock_erp', 'erp', 'inventario')

def _fmt_clp(n: float) -> str:
    try:
        v = int(round(float(n or 0)))
    except (TypeError, ValueError):
        v = 0
    return f"${v:,}".replace(",", ".")

def _color_por_id(color_id: str) -> dict[str, Any] | None:
    if _modo_stock():
        c = stock_palette.color_por_id(color_id)
        if c:
            return c
    return cartilla.color_por_id(color_id)

def _ambiente_por_id(ambiente_id: str) -> dict[str, Any] | None:
    aid = (ambiente_id or "").strip().lower()
    for a in AMBIENTES:
        if a["id"] == aid:
            return dict(a)
    return None

def _brillo_por_id(brillo_id: str) -> dict[str, Any] | None:
    bid = (brillo_id or "").strip().lower()
    for b in BRILLOS:
        if b["id"] == bid:
            return dict(b)
    return None

def familias_colores(*, uso: str = 'interior') -> list[dict[str, Any]]:
    if _modo_stock():
        return stock_palette.familias_desde_stock(uso=uso, solo_con_stock=True)
    return cartilla.familias_colores(uso=uso)

def _paleta_default_id(uso: str = 'interior') -> str:
    fams = familias_colores(uso=uso)
    if fams and fams[0].get('colores'):
        return fams[0]['colores'][0]['id']
    if _modo_stock():
        return ''
    cols = cartilla.paleta_completa()
    return cols[0]['id'] if cols else ''

def payload_inicial(*, uso: str = 'interior') -> dict[str, Any]:
    fams_int = familias_colores(uso='interior')
    fams_ext = familias_colores(uso='exterior')
    fams = fams_ext if uso == 'exterior' else fams_int
    if _modo_stock():
        cols = stock_palette.paleta_desde_stock(uso=uso, solo_con_stock=True)
        meta = stock_palette.resumen_stock()
    else:
        cols = cartilla.paleta_completa(solo_exterior=True if uso == 'exterior' else False)
        meta = cartilla.meta_cartilla()
    sin_stock = _modo_stock() and len(cols) == 0

```

```

return {
    "modo": "stock_erp" if _modo_stock() else "cartilla",
    "ambientes": AMBIENTES,
    "brillos": BRILLOS,
    "calidades": [] if _modo_stock() else CALIDADES,
    "familias": fams,
    "familias_interior": fams_int,
    "familias_exterior": fams_ext,
    "colores": cols,
    "cartilla": meta,
    "sin_stock": sin_stock,
    "defaults": {
        "manos": MANOS_DEFAULT,
        "rendimiento_m2_galon": RENDIMIENTO_M2_GALON,
        "litros_por_galon": LITROS_POR_GALON,
        "color_id": _paleta_default_id(uso=uso),
    },
    "scene_assets": dict(_SCENE_BG),
}

def calcular_cantidad(*, m2: float, manos: int | None = None, rendimiento: float | None = None) -> dict[str, Any]:
    try:
        superficie = max(1.0, float(m2 or 0))
    except (TypeError, ValueError):
        superficie = 1.0
    n_manos = max(1, min(int(manos or MANOS_DEFAULT), 4))
    rend = max(10.0, float(rendimiento or RENDIMIENTO_M2_GALON))
    litros = (superficie * n_manos) / rend * LITROS_POR_GALON
    galones = litros / LITROS_POR_GALON
    galones_ceil = max(1, math.ceil(galones * 4) / 4) # redondeo a 1/4 galón
    return {
        "m2": round(superficie, 1),
        "manos": n_manos,
        "rendimiento_m2_galon": rend,
        "litros_estimados": round(litros, 2),
        "galones_estimados": round(galones, 2),
        "galones_sugeridos": galones_ceil,
        "galones_sugeridos_fmt": f"{galones_ceil:.2f}".replace(".", ","),
    }

def _es_pintura_row(prod) -> bool:
    if stock_palette.es_pintura_con_color(prod):
        return True
    nombre = (prod.nombre or "").lower()
    cat = (prod.categoria or "").lower()
    sub = (prod.subcategoria or "").lower()
    if any(x in nombre for x in ("rodillo", "brocha", "bandeja", "cinta masking", "lija", "thinner",
"diluyente", "cinta", "fibra")):
        return False
    if "pintur" in cat or "pintur" in sub or "esmalte" in nombre or "latex" in nombre or "látex" in nombre:
        return True
    if nombre.startswith("pintura ") or " pintura " in nombre:
        return True
    return False

def _producto_pintura_por_id(producto_id: int) -> dict[str, Any] | None:
    from app import ChilematVtexProducto, Producto
    from services.stock_service import stock_tienda_por_producto_ids

    p = Producto.query.get(int(producto_id))
    if not p or not p.activo:
        return None
    st = stock_tienda_por_producto_ids([p.id]).get(p.id, 0)
    chm = ChilematVtexProducto.query.filter_by(producto_id=p.id).first()

```

```

    if _modo_stock():
        return stock_palette._serializar_item(p, int(st or 0), chm)
    return _serializar_producto_pintura(p, chm, int(st or 0))

def _serializar_producto_pintura(prod, chm=None, stock: int = 0) -> dict[str, Any]:
    precio = float(prod.precio_venta or prod.precio_mayoreo or 0)
    img = (prod.imagen_url or "").strip()
    ref = (prod.codigo_chilemat or prod.codigo_interno or prod.codigo_barra or "").strip()
    marca = (prod.marca or "").strip() if hasattr(prod, "marca") else ""
    if chm:
        if not img:
            img = (getattr(chm, "imagen_url", None) or "").strip()
        if (getattr(chm, "precio_lista", None) or 0) > 0:
            precio = float(chm.precio_lista)
        ref = (chm.product_reference or ref or "").strip()
        marca = (chm.brand or marca or "").strip()
    return {
        "producto_id": prod.id,
        "nombre": (prod.nombre or "")[:120],
        "precio": int(round(precio)),
        "precio_fmt": _fmt_clp(precio),
        "imagen_url": img[:500] if img else None,
        "referencia": ref[:80],
        "marca": marca[:60],
        "stock_tienda": int(stock or 0),
        "disponible": int(stock or 0) > 0,
    }

def productos_pintura_por_calidad(
    *,
    calidad_id: str,
    uso: str = "interior",
    limite: int = 12,
    marca_preferida: str | None = None,
) -> list[dict[str, Any]]:
    from app import ChilematVtexProducto, Producto
    from services.stock_service import stock_tienda_por_producto_ids

    calidad_id = (calidad_id or "standard").strip().lower()
    uso = (uso or "interior").strip().lower()

    q = (
        Producto.query.filter(Producto.activo.is_(True))
        .filter((Producto.precio_venta > 0) | (Producto.precio_mayoreo > 0))
        .order_by(Producto.precio_venta.asc())
    )
    rows = q.limit(400).all()
    candidatos = [p for p in rows if _es_pintura_row(p)]

    if uso == "exterior":
        ext = [
            p
            for p in candidatos
            if "exterior" in (p.nombre or "").lower() or "fachada" in (p.nombre or "").lower()
        ]
        if ext:
            candidatos = ext

    if len(candidatos) < 3:
        candidatos = rows[:80]

    candidatos.sort(key=lambda p: float(p.precio_venta or p.precio_mayoreo or 0))
    marca_pref = (marca_preferida or '').strip().lower()
    if marca_pref:
        con_marca = [
            p for p in candidatos

```

```

        if marca_pref in (getattr(p, 'marca', None) or '').lower()
        or marca_pref in (p.nombre or '').lower()
    ]
    if con_marca:
        candidatos = con_marca + [p for p in candidatos if p not in con_marca]
n = len(candidatos)
if n == 0:
    return []

if calidad_id == "economica":
    slice_rows = candidatos[: max(3, n // 3)]
elif calidad_id == "premium":
    slice_rows = candidatos[max(0, n - max(3, n // 3)) :]
else:
    mid = n // 3
    slice_rows = candidatos[mid : mid + max(3, n // 3)] or candidatos

pids = [p.id for p in slice_rows[:limite]]
stocks = stock_tienda_por_producto_ids(pids) if pids else {}
chm_map: dict[int, Any] = {}
for chm in ChilematVtexProducto.query.filter(ChilematVtexProducto.producto_id.in_(pids)).all():
    if chm.producto_id and int(chm.producto_id) not in chm_map:
        chm_map[int(chm.producto_id)] = chm

out = []
for p in slice_rows[:limite]:
    out.append(_serializar_producto_pintura(p, chm_map.get(p.id), stocks.get(p.id, 0)))
out.sort(key=lambda x: x["precio"])
return out

def complementos_pintura(producto_id: int | None, *, limite: int = 4) -> list[dict[str, Any]]:
    if not producto_id:
        return []
    try:
        from services.producto_relacion_service import sugerencias_para_carrito
        from services.stock_service import stock_tienda_por_producto_ids
        from app import Producto

        raw = sugerencias_para_carrito([int(producto_id)], limite=limite)
        if not raw:
            return []
        pids = [int(x["id"]) for x in raw]
        stocks = stock_tienda_por_producto_ids(pids)
        out = []
        for it in raw:
            p = Producto.query.get(int(it["id"]))
            if not p:
                continue
            precio = int(round(float(it.get("precio") or p.precio_venta or 0)))
            out.append(
                {
                    "producto_id": p.id,
                    "nombre": (it.get("nombre") or p.nombre or "")[:100],
                    "precio": precio,
                    "precio_fmt": _fmt_clp(precio),
                    "stock_tienda": int(stocks.get(p.id, 0)),
                    "disponible": int(stocks.get(p.id, 0)) > 0,
                }
            )
        return out[:limite]
    except Exception:
        return []

def cotizar_proyecto(
    *,
    ambiente_id: str,

```

```

color_id: str,
brillo_id: str,
m2: float,
calidad_id: str = "standard",
producto_id: int | None = None,
) -> dict[str, Any]:
    amb = _ambiente_por_id(ambiente_id) or AMBIENTES[0]
    color = _color_por_id(color_id)
    if not color and _modo_stock():
        color = {'nombre': 'Pintura', 'codigo': '', 'marca': '', 'hex': '#B0BEC5'}
    elif not color:
        color = cartilla.paleta_completa()[0]
    brillo = _brillo_por_id(brillo_id) or BRILLOS[0]
    cant = calcular_cantidad(m2=m2)

    pid_color = stock_palette.producto_id_desde_color(color_id) if _modo_stock() else None
    if pid_color:
        producto_id = pid_color
    elegido = None
    if producto_id:
        raw = _producto_pintura_por_id(int(producto_id))
        if raw:
            elegido = {
                'producto_id': raw['producto_id'],
                'nombre': raw.get('nombre_completo') or raw['nombre'],
                'precio': raw['precio'],
                'precio_fmt': raw['precio_fmt'],
                'imagen_url': raw.get('imagen_url'),
                'referencia': raw.get('referencia') or raw.get('codigo'),
                'marca': raw.get('marca'),
                'stock_tienda': raw.get('stock_tienda', 0),
                'disponible': raw.get('disponible', False),
            }

    productos: list[dict[str, Any]] = []
    if not elegido:
        productos = productos_pintura_por_calidad(
            calidad_id=calidad_id,
            uso=amb.get("uso", "interior"),
            marca_preferida=color.get("marca"),
        )
    if producto_id:
        for p in productos:
            if int(p["producto_id"]) == int(producto_id):
                elegido = p
                break
    if not elegido and productos:
        con_stock = [p for p in productos if p.get("disponible")]
        elegido = (con_stock or productos)[0]

    comps = complementos_pintura(elegido["producto_id"] if elegido else None)

    titulo = f"Proyecto {amb['nombre']} · {color.get('nombre') or 'Pintura'}"
    codigo_ref = color.get('codigo') or (elegido or {}).get('referencia') or ''
    marca_ref = color.get('marca') or (elegido or {}).get('marca') or ''
    resumen = (
        f"{cant['galones_sugeridos_fmt']} gal (≈{cant['m2']} m² × {cant['manos']} manos) · "
        f"{codigo_ref} {marca_ref} · {brillo['nombre']}"
    )

    liz_prompt = (
        f"Estoy configurando pintura para {amb['nombre']}: "
        f"{color.get('nombre') or 'pintura'} ({codigo_ref}), "
        f"acabado {brillo['nombre']}, {cant['m2']} m². ¿Qué más debería llevar?"
    )

    wa_lineas = [
        f"Hola, cotizo pintura – {titulo}",

```

```

        resumen,
    ]
    if elegido:
        wa_lineas.append(f"Base sugerida: {elegido['nombre']} ({elegido['precio_fmt']})")
    wa_lineas.append("Retiro en tienda Santo Domingo.")

    galones_qty = int(cant.get("galones_sugeridos") or 1)
    precio_pintura = int(elegido["precio"]) if elegido else 0
    subtotal_pintura = precio_pintura * galones_qty
    subtotal_comps = sum(int(c.get("precio") or 0) for c in comps)
    total_proyecto = subtotal_pintura + subtotal_comps

    return {
        "ok": True,
        "titulo": titulo,
        "resumen": resumen,
        "ambiente": amb,
        "color": color,
        "brillo": brillo,
        "cantidad": cant,
        "calidad_id": calidad_id,
        "producto": elegido,
        "productos_alternativas": productos[:6],
        "complementos": comps,
        "liz_prompt": liz_prompt,
        "mensaje_whatsapp": "\n".join(wa_lineas),
        "modo": "stock_erp" if _modo_stock() else "cartilla",
        "totales": {
            "galones": galones_qty,
            "subtotal_pintura": subtotal_pintura,
            "subtotal_pintura_fmt": _fmt_clp(subtotal_pintura),
            "subtotal_complementos": subtotal_comps,
            "subtotal_complementos_fmt": _fmt_clp(subtotal_comps),
            "total_proyecto": total_proyecto,
            "total_proyecto_fmt": _fmt_clp(total_proyecto),
        },
        "tinte": {
            "codigo": codigo_ref,
            "nombre": color.get("nombre"),
            "marca": marca_ref,
            "hex": color.get("hex"),
            "modo": "stock_erp" if _modo_stock() else "cartilla",
        },
        "bases_erp": [] if _modo_stock() else cartilla.bases_pintura_erp(marca=color.get("marca"), limite=4),
    }

def _tip_reglas(
    *,
    step: int,
    ambiente_id: str,
    color_id: str,
    brillo_id: str,
    m2: float,
) -> str:
    amb = _ambiente_por_id(ambiente_id) or AMBIENTES[0]
    color = _color_por_id(color_id)
    brillo = _brillo_por_id(brillo_id)
    try:
        superficie = max(1.0, float(m2 or 0))
    except (TypeError, ValueError):
        superficie = 12.0

    if step == 1:
        if amb.get('id') == 'bano':
            return 'En baños recomiendo satinado: aguanta humedad y se limpia con paño húmedo.'
        if amb.get('id') == 'fachada':
            return 'Para fachada elija colores marcados para exterior y pintura con filtro UV.'

```



```

        if amb.get('id') == 'dormitorio':
            return 'Tonos suaves y mate ayudan a un ambiente de descanso.'
        return amb.get('tip') or 'Elija el ambiente y vea la vista previa con su color.'

    if step == 2:
        if amb.get('id') == 'bano' and brillo_id == 'mate':
            return 'En baño el mate puede marcar manchas. Satinado es la opción más usada en ferretería.'
        if amb.get('id') == 'cocina' and brillo_id == 'mate':
            return 'Cocina + satinado = menos grasa adherida. Fácil de pasar paño después de cocinar.'
        if color and color.get('exterior') and amb.get('uso') != 'exterior':
            return f'{color.get("codigo")} es tono exterior; para interior elija otro de la cartilla.'
        if brillo:
            return brillo.get('ideal', '') + '. Cambie brillo y mire la vista previa arriba.'
        if _modo_stock():
            return 'Elija un color de los que hay hoy en tienda. Cada muestra es un producto con stock real.'
        return 'Combine color Kölor/Topex con el brillo según el uso del espacio.'

    if step == 3:
        cant = calcular_cantidad(m2=superficie)
        if superficie <= 10:
            return f'~{cant["galones_sugeridos_fmt"]} galones para {superficie} m². Típico de dormitorio
pequeño.'
        if superficie >= 25:
            return f'Proyecto grande: {cant["galones_sugeridos_fmt"]} gal estimados. Considere margen para
repasos.'
        return f'Con {superficie} m² calculamos {cant["galones_sugeridos_fmt"]} gal a 2 manos. Ajuste si incluye
cielo.'

    return ''

def _sanitizar_tip(txt: str) -> str:
    t = re.sub(r'\s+', ' ', (txt or '').strip())
    t = t.replace('\n', ' ')
    if len(t) > 320:
        t = t[:317].rstrip() + '...'
    return t

def liz_tip_wizard(
    *,
    step: int,
    ambiente_id: str,
    color_id: str,
    brillo_id: str,
    m2: float,
) -> dict[str, Any]:
    """Tip corto Liz – Ollama vitrina con fallback reglas SD."""
    regla = _tip_reglas(
        step=step,
        ambiente_id=ambiente_id,
        color_id=color_id,
        brillo_id=brillo_id,
        m2=m2,
    )
    if step >= 4 or step < 1:
        return {'ok': True, 'tip': '', 'fuente': 'reglas'}

    amb = _ambiente_por_id(ambiente_id) or AMBIENTES[0]
    color = _color_por_id(color_id)
    brillo = _brillo_por_id(brillo_id)
    cant = calcular_cantidad(m2=m2)

    try:
        from services.ollama_client import generar_chat_vitrina, ollama_disponible_vitrina

        if ollama_disponible_vitrina(requiere_modelo=False):
            system = (

```

```

'Eres Liz, asesora de pinturas en ferretería Santo Domingo (Chile). '
'Responde UN solo consejo práctico en español chileno, máximo 2 oraciones, sin saludo. '
'No inventes precios ni stock. Solo recomienda según ambiente, brillo y m². '
'Los colores mostrados ya son productos con stock en tienda (no tintometría aún).'
)
user = (
    f'Paso wizard {step}/4. Ambiente: {amb.get("nombre")}. '
    f'Color: {color.get("codigo") if color else "?"} {color.get("nombre") if color else ""} '
    f'({color.get("marca") if color else ""}). Brillo: {brillo.get("nombre") if brillo else "?"}. '
    f'Superficie: {cant.get("m2")} m², ~{cant.get("galones_sugeridos_fmt")} galones.'
)
out = generar_chat_vitrina(system=system, user=user, timeout=25)
if out.get('ok') and out.get('texto'):
    return {'ok': True, 'tip': _sanitizar_tip(out['texto']), 'fuente': 'ollama'}
except Exception:
    pass

return {'ok': True, 'tip': regla, 'fuente': 'reglas'}

```

services/pintura_stock_palette_service.py

```

"""Paleta Fábrica de Color desde stock ERP (fase 1 – sin tintometría)."""
from __future__ import annotations

import re
import unicodedata
from typing import Any

# familia → (palabras clave en nombre, hex visualizador)
_REGLAS_COLOR: list[tuple[str, tuple[str, ...], str]] = [
    ('blanco', ('blanco', 'ng gl', ' ng '), '#F5F5F5'),
    ('beige', ('beige', 'crema', 'hueso', 'lino', 'arena', 'damasco'), '#EFEBE9'),
    ('amarillo', ('amarillo', 'oro', 'maiz', 'limon'), '#FFEB3B'),
    ('verde', ('verde',), '#4CAF50'),
    ('azul', ('azul', 'celeste', 'piscina'), '#42A5F5'),
    ('gris', ('gris', 'galvanizado', 'zinc'), '#9E9E9E'),
    ('rojo', ('rojo', 'colonial', 'terracota', 'ladrillo', 'coral'), '#E53935'),
    ('neutro', ('negro', ' ng ', 'ng ', 'calorkote ng', 'cafe', 'moro', 'ocre', 'roble', 'pajarito'),
    '#455A64'),
]

_FAMILIAS_ORDEN = ('blanco', 'beige', 'amarillo', 'verde', 'azul', 'gris', 'rojo', 'neutro')

_EXCLUIR_NOMBRE = (
    'rodillo', 'brocha', 'bandeja', 'thinner', 'diluyente', 'lija', 'cinta',
    'fibra de vidrio', 'masking', 'pincheleta', 'espatula', 'aerosol',
    'acido', 'muriatico', 'cloro', 'hipoclorito', 'desincrust', 'detergente',
    'antihongo', 'antimoho', 'sellador', 'impermeabil', 'barniz marino',
)

_EXCLUIR_CATEGORIA_ACC = ('brocha', 'rodillo', 'accesorio')

def _norm_txt(s: str) -> str:
    t = unicodedata.normalize('NFKD', (s or ''))
    t = ''.join(c for c in t if not unicodedata.combining(c))
    return t.lower()

def es_pintura_con_color(prod) -> bool:
    """SKU vendible como color (latex/esmalte/oleo/pintura), no accesorio."""
    nombre = _norm_txt(prod.nombre or '')
    cat = _norm_txt(prod.categoria or '')
    sub = _norm_txt(prod.subcategoria or '')

```

```

    if any(x in nombre for x in _EXCLUIR_NOMBRE):
        return False
    if any(x in sub for x in _EXCLUIR_CATEGORIA_ACC):
        return False
    if 'pintur' in cat or 'pintur' in sub or 'piso y pared' in cat:
        pass
    elif not any(x in nombre for x in ('latex', 'látex', 'esmalte', 'oleo', 'óleo', 'pintura',
'anticorrosivo')):
        return False
    return any(
        kw in nombre
        for _fam, kws, _hx in _REGLAS_COLOR
        for kw in kws
    ) or 'blanco' in nombre or 'color' in nombre

def _clasificar_color(nombre: str) -> tuple[str, str, str]:
    n = _norm_txt(nombre)
    for familia, keywords, hexv in _REGLAS_COLOR:
        for kw in keywords:
            if kw in n:
                label = _label_desde_nombre(nombre, familia)
                return familia, label, hexv
    return 'neutro', (nombre or 'Pintura')[:40], '#B0BEC5'

def _label_desde_nombre(nombre: str, familia: str) -> str:
    raw = (nombre or '').strip()
    if len(raw) <= 42:
        return raw
    n = _norm_txt(nombre)
    for _fam, keywords, _hx in _REGLAS_COLOR:
        for kw in keywords:
            if kw in n:
                idx = n.find(kw)
                # tomar ventana alrededor de la palabra color
                parts = re.split(r'\s+', raw)
                for i, p in enumerate(parts):
                    if kw in _norm_txt(p):
                        start = max(0, i - 1)
                        return ' '.join(parts[start : i + 2])[:42]
                return kw.capitalize()
    return raw[:42]

def _fmt_clp(n: float) -> str:
    try:
        v = int(round(float(n or 0)))
    except (TypeError, ValueError):
        v = 0
    return f'${v:,}'.replace(',', ' ')

def _serializar_item(prod, stock: int, chm=None) -> dict[str, Any]:
    from services.fabrica_color_service import _serializar_producto_pintura

    base = _serializar_producto_pintura(prod, chm, stock)
    fam, label, hexv = _clasificar_color(prod.nombre or '')
    base.update({
        'id': f'p{prod.id}',
        'familia': fam,
        'nombre': label,
        'nombre_completo': (prod.nombre or '')[:120],
        'hex': hexv,
        'codigo': (prod.codigo_chilemat or prod.codigo_interno or prod.codigo_barra or str(prod.id))[:40],
        'marca': base.get('marca') or _marca_desde_nombre(prod.nombre or ''),
    })
    return base

```

```

def _marca_desde_nombre(nombre: str) -> str:
    n = nombre or ''
    for m in ('Ceresita', 'Soquina', 'Sipa', 'Sherwin', 'Kolor', 'Topex', 'Romeral'):
        if m.lower() in n.lower():
            return m
    return ''

def _filtro_uso(nombre: str, uso: str) -> bool:
    n = _norm_txt(nombre)
    if uso == 'exterior':
        return any(x in n for x in ('exterior', 'fachada', 'anticorrosivo', 'galvanizado', 'piscina'))
    if uso == 'interior':
        if any(x in n for x in ('piscina', 'galvanizado', 'anticorrosivo')):
            return False
    return True

def paleta_desde_stock(*, uso: str = 'interior', solo_con_stock: bool = True) -> list[dict[str, Any]]:
    from app import ChilematVtexProducto, Producto
    from services.stock_service import stock_tienda_por_producto_ids

    uso = (uso or 'interior').strip().lower()
    q = (
        Producto.query.filter(Producto.activo.is_(True))
        .filter((Producto.precio_venta > 0) | (Producto.precio_mayoreo > 0))
    )
    rows = q.limit(3000).all()
    candidatos = [p for p in rows if es_pintura_con_color(p) and _filtro_uso(p.nombre or '', uso)]
    pids = [p.id for p in candidatos]
    stocks = stock_tienda_por_producto_ids(pids) if pids else {}

    if solo_con_stock:
        candidatos = [p for p in candidatos if int(stocks.get(p.id, 0) or 0) > 0]

    candidatos.sort(key=lambda p: (-int(stocks.get(p.id, 0) or 0), p.nombre or ''))

    chm_map: dict[int, Any] = {}
    if pids:
        for chm in ChilematVtexProducto.query.filter(ChilematVtexProducto.producto_id.in_(pids)).all():
            if chm.producto_id and int(chm.producto_id) not in chm_map:
                chm_map[int(chm.producto_id)] = chm

    out = []
    for p in candidatos:
        st = int(stocks.get(p.id, 0) or 0)
        out.append(_serializar_item(p, st, chm_map.get(p.id)))
    return out

def familias_desde_stock(*, uso: str = 'interior', solo_con_stock: bool = True) -> list[dict[str, Any]]:
    cols = paleta_desde_stock(uso=uso, solo_con_stock=solo_con_stock)
    out: list[dict[str, Any]] = []
    for fam in _FAMILIAS_ORDEN:
        fam_cols = [c for c in cols if c.get('familia') == fam]
        if fam_cols:
            out.append({'id': fam, 'nombre': fam.capitalize(), 'colores': fam_cols})
    # familia "otros" si quedaron sin clasificar
    usados = {c['id'] for f in out for c in f.get('colores', [])}
    otros = [c for c in cols if c['id'] not in usados]
    if otros:
        out.append({'id': 'otros', 'nombre': 'Otros', 'colores': otros})
    return out

def color_por_id(color_id: str) -> dict[str, Any] | None:

```

```

cid = (color_id or '').strip().lower()
if cid.startswith('p') and cid[1:].isdigit():
    pid = int(cid[1:])
    from app import Producto
    from services.stock_service import stock_tienda_por_producto_ids

    p = Producto.query.get(pid)
    if not p or not es_pintura_con_color(p):
        return None
    st = stock_tienda_por_producto_ids([pid]).get(pid, 0)
    return _serializar_item(p, int(st or 0))
return None

def producto_id_desde_color(color_id: str) -> int | None:
    cid = (color_id or '').strip().lower()
    if cid.startswith('p') and cid[1:].isdigit():
        return int(cid[1:])
    c = color_por_id(cid)
    if c and c.get('producto_id'):
        return int(c['producto_id'])
    return None

def resumen_stock() -> dict[str, Any]:
    int_cols = paleta_desde_stock(uso='interior', solo_con_stock=True)
    ext_cols = paleta_desde_stock(uso='exterior', solo_con_stock=True)
    return {
        'modo': 'stock_erp',
        'total_interior': len(int_cols),
        'total_exterior': len(ext_cols),
        'total': len({c['producto_id'] for c in int_cols + ext_cols}),
        'solo_stock_tienda': True,
    }

```

services/modulo_pinturas_session_service.py

```

"""
Sesiones del módulo cliente Fábrica de Color (pinturas).
Fase A: preview lab vía env. Fase B: token firmado emitido desde caja/POS.
"""

from __future__ import annotations

import os
from datetime import datetime, timedelta, timezone
from typing import Any

from itsdangerous import BadSignature, SignatureExpired, URLSafeTimedSerializer

LAB_TOKEN = 'lab'
SERIALIZER_SALT = 'modulo-pinturas-v1'
DEFAULT_TTL_MINUTES = 20

def preview_habilitado() -> bool:
    return (os.getenv('VITRINA_FABRICA_COLOR_PREVIEW', '0')).strip().lower() in ('1', 'true', 'si', 'yes', 'on'))

def ttl_minutos() -> int:
    try:
        return max(5, min(int(os.getenv('MODULO_PINTURAS_TTL_MIN', str(DEFAULT_TTL_MINUTES))), 120))
    except (TypeError, ValueError):
        return DEFAULT_TTL_MINUTES

```

```

def _serializer():
    from flask import current_app

    return URLSafeTimedSerializer(current_app.secret_key, salt=SERIALIZER_SALT)

def crear_sesion_modulo_pinturas(*, usuario_id: int, usuario_nombre: str) -> dict[str, Any]:
    ttl = ttl_minutos()
    payload = {
        'uid': int(usuario_id),
        'nombre': (usuario_nombre or 'Mostrador')[:80],
        'mod': 'fabrica_color',
    }
    token = _serializer().dumps(payload)
    exp = datetime.now(timezone.utc) + timedelta(minutes=ttl)
    path = f'/modulos/pinturas/{token}'
    return {
        'ok': True,
        'token': token,
        'path': path,
        'modulo': 'fabrica_color',
        'ttl_minutos': ttl,
        'expires_at': exp.isoformat(),
        'habilitado_por': payload['nombre'],
    }

def validar_acceso(token: str) -> dict[str, Any] | None:
    t = (token or '').strip()
    if not t:
        return None
    if t == LAB_TOKEN:
        if preview_habilitado():
            return {
                'modo': 'lab',
                'uid': 0,
                'nombre': 'Preview lab',
                'mod': 'fabrica_color',
            }
        return None
    try:
        data = _serializer().loads(t, max_age=ttl_minutos() * 60)
        if not isinstance(data, dict):
            return None
        return {
            'modo': 'caja',
            'uid': int(data.get('uid') or 0),
            'nombre': (data.get('nombre') or 'Mostrador')[:80],
            'mod': data.get('mod') or 'fabrica_color',
        }
    except (BadSignature, SignatureExpired, TypeError, ValueError):
        return None

def acceso_permitido(token: str) -> bool:
    return validar_acceso(token) is not None

```

services/pintura_cartilla_service.py

```

"""Cartilla Kölor/Topex – paleta Fábrica de Color (JSON + vínculo opcional ERP)."""
from __future__ import annotations

import json

```

```

import os
import re
from functools import lru_cache
from typing import Any

_FAMILIAS_ORDEN = ('blanco', 'beige', 'amarillo', 'verde', 'azul', 'gris', 'rojo', 'neutro')

def _repo_root() -> str:
    return os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))

def _cartilla_path() -> str:
    override = (os.getenv('PINTURA_CARTILLA_JSON') or '').strip()
    if override and os.path.isfile(override):
        return override
    return os.path.join(_repo_root(), 'data', 'pintura_cartilla_sd.json')

def _slug_codigo(codigo: str) -> str:
    return re.sub(r'^a-z0-9+', '-', (codigo or '').strip().lower()).strip('-')

@lru_cache(maxsize=1)
def _raw_cartilla() -> dict[str, Any]:
    path = _cartilla_path()
    try:
        with open(path, encoding='utf-8') as fh:
            data = json.load(fh)
            if isinstance(data, dict) and isinstance(data.get('colores'), list):
                return data
    except OSError:
        pass
    except json.JSONDecodeError:
        pass
    return {'colores': [], 'version': 'fallback'}

def _normalizar_color(row: dict[str, Any]) -> dict[str, Any]:
    codigo = (row.get('codigo') or '').strip().upper()
    familia = (row.get('familia') or 'neutro').strip().lower()
    if familia not in _FAMILIAS_ORDEN:
        familia = 'neutro'
    return {
        'id': _slug_codigo(codigo),
        'codigo': codigo,
        'nombre': (row.get('nombre') or codigo).strip()[:80],
        'familia': familia,
        'hex': (row.get('hex') or '#CCCCC').strip(),
        'marca': (row.get('marca') or 'Kolor').strip()[:40],
        'exterior': bool(row.get('exterior')),
    }

def paleta_completa(*, solo_exterior: bool | None = None) -> list[dict[str, Any]]:
    rows = [_normalizar_color(r) for r in (_raw_cartilla().get('colores') or []) if isinstance(r, dict)]
    if solo_exterior is True:
        rows = [c for c in rows if c.get('exterior')]
    elif solo_exterior is False:
        rows = [c for c in rows if not c.get('exterior')]
    return rows

def color_por_id(color_id: str) -> dict[str, Any] | None:
    cid = (color_id or '').strip().lower()
    if not cid:
        return None
    for c in paleta_completa():

```

```

        if c['id'] == cid:
            return dict(c)
    return None

def color_por_codigo(codigo: str) -> dict[str, Any] | None:
    cod = (codigo or '').strip().upper()
    for c in paleta_completa():
        if c.get('codigo') == cod:
            return dict(c)
    return None

def familias_colores(*, uso: str = 'interior') -> list[dict[str, Any]]:
    uso = (uso or 'interior').strip().lower()
    cols = list(paleta_completa())
    if uso == 'exterior':
        cols.sort(
            key=lambda c: (
                0 if c.get('exterior') else 1,
                _FAMILIAS_ORDEN.index(c['familia']) if c.get('familia') in _FAMILIAS_ORDEN else 99,
                c.get('codigo') or '',
            )
        )
    out: list[dict[str, Any]] = []
    for fam in _FAMILIAS_ORDEN:
        fam_cols = [c for c in cols if c.get('familia') == fam]
        if fam_cols:
            out.append({'id': fam, 'nombre': fam.capitalize(), 'colores': fam_cols})
    return out

def meta_cartilla() -> dict[str, Any]:
    raw = _raw_cartilla()
    cols = paleta_completa()
    return {
        'version': raw.get('version') or '',
        'total_colores': len(cols),
        'marcas': sorted({c.get('marca') for c in cols if c.get('marca')}),
        'fuente': os.path.basename(_cartilla_path()),
    }

def bases_pintura_erp(*, marca: str | None = None, limite: int = 8) -> list[dict[str, Any]]:
    """Bases latex/esmalte en ERP que coinciden con marca cartilla (Kolor/Topex)."""
    try:
        from app import Producto
        from services.stock_service import stock_tienda_por_producto_ids

        marca_q = (marca or '').strip().lower()
        q = (
            Producto.query.filter(Producto.activo.is_(True))
            .filter((Producto.precio_venta > 0) | (Producto.precio_mayoreo > 0))
            .order_by(Producto.precio_venta.asc())
        )
        rows = q.limit(500).all()
        out = []
        for p in rows:
            nombre = (p.nombre or '').lower()
            cat = (p.categoria or '').lower()
            if any(x in nombre for x in ('rodillo', 'brocha', 'thinner', 'diluyente', 'cinta', 'lija')):
                continue
            if not ('pintur' in cat or 'latex' in nombre or 'esmalte' in nombre or 'látex' in nombre):
                continue
            pm = (getattr(p, 'marca', None) or '').lower()
            if marca_q and marca_q not in pm and marca_q not in nombre:
                continue
            out.append(p)
    
```



```

        if len(out) >= limite * 3:
            break
        pids = [p.id for p in out[:limite]]
        stocks = stock_tienda_por_producto_ids(pids) if pids else {}
        serial = []
        for p in out[:limite]:
            serial.append({
                'producto_id': p.id,
                'nombre': (p.nombre or '')[:100],
                'marca': (getattr(p, 'marca', None) or '')[:40],
                'precio': int(round(float(p.precio_venta or p.precio_mayoreo or 0))),
                'stock_tienda': int(stocks.get(p.id, 0)),
            })
        return serial
    except Exception:
        return []

```

templates/modulos/fabrica_color.html

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="{{ page_meta_description }}">
    <meta name="robots" content="noindex, nofollow">
    <link rel="canonical" href="{{ page_canonical }}">
    <title>Colores en tienda - {{ nombre_tienda }}</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/bootstrap.min.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='vendor/fontawesome/css/all.min.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/tienda-vitrina.css') }}"?v=tienda-shell-16-
chilemat-holding">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/fabrica-color.css') }}"?v=fc-modulo-20260530k">
</head>
<body class="tienda-vitrina fabrica-color-page mp-page{% if tv_mode %} fc-tv-mode{% endif %}">
    {% if es_preview_lab %}
    <div class="mp-banner mp-banner--lab" role="status">
        <i class="fas fa-flask me-2"></i> Preview lab - no publicado - solo desarrollo / SAMBOX
    </div>
    {% elif sesion_modulo and sesion_modulo.modulo == 'caja' %}
    <div class="mp-banner mp-banner--caja" role="status">
        <i class="fas fa-user-check me-2"></i> Sesión habilitada por mostrador
        {% if sesion_modulo.nombre %} · {{ sesion_modulo.nombre }}{% endif %}
    </div>
    {% endif %}

    {% set mp_home_url = fabrica_config.modulo_home_url %}
    {% include 'modulos/includes/header_modulo_pinturas.html' %}

    <main class="fc-main" id="fcMain" data-step="1">
        <header class="fc-top">
            <div class="fc-top__copy">
                <span class="fc-kicker">LhexIA · Asistente pinturas</span>
                <h1>Colores en tienda</h1>
                {% if fabrica_config.modulo_home_url %}
                <a href="{{ fabrica_config.modulo_home_url }}" class="fc-link-back"><i class="fas fa-arrow-left me-
1"></i> Volver al lab</a>
                {% endif %}
            </div>
        </header>

        <div class="fc-progress" aria-label="Progreso del asistente">
            <div class="fc-progress__bar"><span id="fcProgressFill"></span></div>
            <ol class="fc-steps-nav" id="fcStepsNav">

```

```

        <li class="is-active" data-step="1"><span>1</span> Ambiente</li>
        <li data-step="2"><span>2</span> Color</li>
        <li data-step="3"><span>3</span> Cantidad</li>
        <li data-step="4"><span>4</span> Resumen</li>
    </ol>
</div>

<section class="fc-pick is-active" id="fcPick" data-step="1">
    <h2 class="fc-pick__title">Elige el ambiente que deseas pintar</h2>
    <p class="fc-pick__lead">Toque una foto para ver colores con stock real en tienda.</p>
    <div class="fc-amb-grid" id="fcAmbientes"></div>
</section>

<div class="fc-workspace" id="fcWorkspace" hidden>
    <div class="fc-workspace__preview">
        <div class="fc-scene fc-scene--canvas" id="fcScene" data-scene="interior" aria-label="Vista previa del
ambiente">
            <canvas class="fc-scene__canvas" id="fcSceneCanvas"></canvas>
            <div class="fc-scene__swatch" id="fcSceneSwatch"></div>
            <div class="fc-scene__label" id="fcSceneLabel">Vista previa</div>
            <p class="fc-scene__disclaimer">Vista referencial · solo muro principal</p>
        </div>
        <button type="button" class="fc-change-ambiente" id="fcChangeAmbiente">
            <i class="fas fa-chevron-up me-1"></i> Mostrar otros ambientes
        </button>
    </div>

    <div class="fc-workspace__panel">
        <div class="fc-wizard">
            <section class="fc-step" data-step="2">
                <h2>Color disponible en tienda</h2>
                <p class="fc-step-lead">Solo mostramos pinturas con <strong>stock en tienda hoy</strong> (<span
id="fcCartillaCount">0</span> opciones).</p>
                <div class="fc-panel-section">
                    <h3 class="fc-panel-heading">Colores</h3>
                    <div class="fc-familias" id="fcFamilias"></div>
                    <div class="fc-paleta" id="fcPaleta"></div>
                </div>
                <div class="fc-panel-section">
                    <h3 class="fc-panel-heading">Brillo</h3>
                    <div class="fc-brillos" id="fcBrillos"></div>
                </div>
            </section>

            <section class="fc-step" data-step="3">
                <h2>Calcular cantidad</h2>
                <p class="fc-step-lead">Ingrese metros cuadrados a pintar (muro). Estimamos galones a 2 manos.</p>
                <div class="fc-cantidad-form">
                    <label for="fcM2">Superficie (m²)</label>
                    <div class="fc-m2-row">
                        <input type="number" id="fcM2" min="1" max="500" step="0.5" value="12" inputmode="decimal">
                        <button type="button" class="fc-btn fc-btn--ghost" data-m2="8">8 m²</button>
                        <button type="button" class="fc-btn fc-btn--ghost" data-m2="15">15 m²</button>
                        <button type="button" class="fc-btn fc-btn--ghost" data-m2="25">25 m²</button>
                    </div>
                </div>
                <div class="fc-cantidad-resumen" id="fcCantidadResumen"></div>
            </section>

            <section class="fc-step" data-step="4">
                <h2>Su proyecto</h2>
                <p class="fc-step-lead" id="fcStep4Lead">Producto en stock, cantidad estimada y complementos
sugeridos.</p>
                <div class="fc-calidades d-none" id="fcCalidades"></div>
                <div class="fc-resultado fc-resultado--hero" id="fcResultado">
                    <p class="text-muted mb-0">Calculando su proyecto...</p>
                </div>
                <div class="fc-complementos" id="fcComplementos"></div>
            </section>
        </div>
    </div>

```

```

        <div class="fc-proyecto-total d-none" id="fcProyectoTotal"></div>
        <div class="fc-acciones" id="fcAcciones"></div>
    </section>
</div>

<nav class="fc-nav">
    <button type="button" class="fc-btn fc-btn--ghost" id="fcPrev" disabled>Anterior</button>
    <button type="button" class="fc-btn fc-btn--primary" id="fcNext">Siguiente</button>
</nav>
</div>
</div>

<p class="tienda-legal fc-legal">
    Colores según stock ERP en tienda. Precios referenciales. Cobro y despacho final en caja.
</p>
</main>

<script id="fabricaColorConfig" type="application/json">{{ fabrica_config|tojson }}</script>
{% include 'tienda/includes/carrito_vitrina_shell.html' %}
{% include 'tienda/includes/liz_asistente.html' %}
<script src="{{ url_for('static', filename='js/tienda-vitrina.js') }}"?v=liz-vitrina-17-fix-500"></script>
<script src="{{ url_for('static', filename='js/fabrica-color.js') }}"?v=fc-modulo-20260530k"></script>
</body>
</html>

```

templates/modulos/pinturas_lab.html

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="robots" content="noindex, nofollow">
    <title>Lab · Módulo pinturas - {{ nombre_tienda }}</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='css/bootstrap.min.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='vendor/fontawesome/css/all.min.css') }}">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/fabrica-color.css') }}"?v=fc-modulo-20260530e">
</head>
<body class="mp-lab-page">
    <div class="mp-banner mp-banner--lab"><i class="fas fa-flask me-2"></i> Preview lab · módulo pinturas (no
indexar)</div>
    <main class="mp-lab-main">
        
        <h1>Colores en tienda</h1>
        <p class="lead">Asistente de pinturas con stock ERP. En piso, el cajero habilita la pantalla para el cliente
desde caja.</p>
        <div class="d-flex flex-wrap gap-2 mt-4">
            <a href="{{ wizard_url }}" class="fc-btn fc-btn--primary text-decoration-none">
                <i class="fas fa-palette me-1"></i> Abrir wizard (lab)
            </a>
            <a href="{{ vitrina_url }}" class="fc-btn fc-btn--ghost text-decoration-none">Vitrina catálogo</a>
        </div>
        <p class="mp-lab-hint mt-4">
            Requiere <code>VITRINA_FABRICA_COLOR_PREVIEW=1</code> en el servidor.
            Producción: el cliente accede solo con URL emitida por <code>POST /api/caja/modulo-
pinturas/habilitar</code>.
        </p>
    </main>
</body>
</html>

```

templates/modulos/includes/header_modulo_pinturas.html

```
<header class="mp-top">
  <div class="mp-top-inner">
    <a href="{ { mp_home_url or '#' } }" class="mp-brand text-decoration-none">
      
      <span>
        <strong>Colores en tienda</strong>
        <small>{ { nombre_tienda } }</small>
      </span>
    </a>
    { % if wa_base % }
    <a class="btn-wa mp-wa" href="{ { wa_base } }" target="_blank" rel="noopener"><i class="fab fa-
whatsapp"></i></a>
    { % endif % }
  </div>
</header>
```

static/js/fabrica-color.js

```
(function () {
  'use strict';

  var cfgEl = document.getElementById('fabricaColorConfig');
  if (!cfgEl) return;

  var cfg;
  try {
    cfg = JSON.parse(cfgEl.textContent || '{}');
  } catch (e) {
    return;
  }

  var initial = cfg.initial || {};
  var stockMode = initial.modos === 'stock_erp';
  var sceneUrIs = cfg.scene_urls || {};
  var familiasInterior = initial.familias_interior || initial.familias || [];
  var familiasExterior = initial.familias_exterior || [];
  var lizTipTimer = null;
  var lizTipSeq = 0;
  var sceneImageCache = {};
  var sceneDrawToken = 0;
  var sceneResizeTimer = null;
  var DEFAULT_WALL_POLYGON = [[0.05, 0.08], [0.95, 0.08], [0.95, 0.55], [0.05, 0.55]];

  var state = {
    step: 1,
    ambiente_id: (initial.ambientes && initial.ambientes[0] && initial.ambientes[0].id) || 'comedor',
    color_id: (initial.defaults && initial.defaults.color_id) || (initial.colores && initial.colores[0] &&
initial.colores[0].id) || '',
    brillo_id: (initial.brillos && initial.brillos[0] && initial.brillos[0].id) || 'mate',
    familia_id: (initial.familias && initial.familias[0] && initial.familias[0].id) || 'blanco',
    m2: 12,
    calidad_id: 'standard',
    producto_id: null,
    cotizacion: null,
  };

  var els = {
    main: document.getElementById('fcMain'),
    pick: document.getElementById('fcPick'),
  };
```

```

workspace: document.getElementById('fcWorkspace'),
changeAmbiente: document.getElementById('fcChangeAmbiente'),
scene: document.getElementById('fcScene'),
sceneCanvas: document.getElementById('fcSceneCanvas'),
sceneSwatch: document.getElementById('fcSceneSwatch'),
sceneLabel: document.getElementById('fcSceneLabel'),
lizTip: null,
lizTipText: null,
cartillaCount: document.getElementById('fcCartillaCount'),
progressFill: document.getElementById('fcProgressFill'),
stepsNav: document.getElementById('fcStepsNav'),
ambientes: document.getElementById('fcAmbientes'),
familias: document.getElementById('fcFamilias'),
paleta: document.getElementById('fcPaleta'),
brillos: document.getElementById('fcBrillos'),
m2: document.getElementById('fcM2'),
cantidadResumen: document.getElementById('fcCantidadResumen'),
calidades: document.getElementById('fcCalidades'),
resultado: document.getElementById('fcResultado'),
complementos: document.getElementById('fcComplementos'),
proyectoTotal: document.getElementById('fcProyectoTotal'),
acciones: document.getElementById('fcAcciones'),
prev: document.getElementById('fcPrev'),
next: document.getElementById('fcNext'),
};

function activeFamilias() {
  var amb = ambienteById(state.ambiente_id);
  if (amb && amb.uso === 'exterior' && familiasExterior.length) {
    return familiasExterior;
  }
  return familiasInterior.length ? familiasInterior : (initial.familias || []);
}

function allActiveColores() {
  var fams = activeFamilias();
  var out = [];
  fams.forEach(function (f) {
    (f.colores || []).forEach(function (c) { out.push(c); });
  });
  return out;
}

function colorById(id) {
  var cols = allActiveColores().concat(initial.colores || []);
  return cols.find(function (c) { return c.id === id; })
    || (initial.colores || []).find(function (c) { return c.id === id; });
}

function ambienteById(id) {
  return (initial.ambientes || []).find(function (a) { return a.id === id; });
}

function brilloById(id) {
  return (initial.brillos || []).find(function (b) { return b.id === id; });
}

function sceneForAmbiente(amb) {
  return (amb && amb.scene) || 'interior';
}

function calcLocalCantidad() {
  var m2 = parseFloat(state.m2) || 1;
  var manos = (initial.defaults && initial.defaults.manos) || 2;
  var rend = (initial.defaults && initial.defaults.rendimiento_m2_galon) || 35;
  var litrosPorGalon = (initial.defaults && initial.defaults.litros_por_galon) || 3.785;
  var litros = (m2 * manos) / rend * litrosPorGalon;
  var galones = litros / litrosPorGalon;
}

```

```

var galonesCeil = Math.max(1, Math.ceil(galones * 4) / 4);
return {
  m2: m2,
  manos: manos,
  galones_sugeridos: galonesCeil,
  galones_sugeridos_fmt: galonesCeil.toFixed(2).replace('.', ','),
};
}

function pulseScene() {
  if (!els.scene) return;
  els.scene.classList.remove('is-updating');
  void els.scene.offsetWidth;
  els.scene.classList.add('is-updating');
  setTimeout(function () {
    if (els.scene) els.scene.classList.remove('is-updating');
  }, 520);
}

function loadSceneImage(url) {
  if (!url) return Promise.reject(new Error('no_url'));
  if (sceneImageCache[url] && sceneImageCache[url].complete && sceneImageCache[url].naturalWidth) {
    return Promise.resolve(sceneImageCache[url]);
  }
  return new Promise(function (resolve, reject) {
    var img = new Image();
    img.crossOrigin = 'anonymous';
    img.onload = function () {
      sceneImageCache[url] = img;
      resolve(img);
    };
    img.onerror = reject;
    img.src = url;
  });
}

function wallPolygonFor(amb) {
  if (amb && amb.wall_polygons && amb.wall_polygons[0] && amb.wall_polygons[0].length >= 3) {
    return amb.wall_polygons[0];
  }
  if (amb && amb.wall_polygon && amb.wall_polygon.length >= 3) {
    return amb.wall_polygon;
  }
  return DEFAULT_WALL_POLYGON;
}

function traceWallPolygon(ctx, polygon, w, h) {
  ctx.beginPath();
  polygon.forEach(function (pt, i) {
    var x = pt[0] * w;
    var y = pt[1] * h;
    if (i === 0) ctx.moveTo(x, y);
    else ctx.lineTo(x, y);
  });
  ctx.closePath();
}

function brilloPaintAlpha(brilloId) {
  if (brilloId === 'semi_brillo') return 0.68;
  if (brilloId === 'satinado') return 0.58;
  return 0.52;
}

function applyMaskTint(ctx, w, h, hex, brilloId, maskImg) {
  var off = document.createElement('canvas');
  off.width = w;
  off.height = h;
  var o = off.getContext('2d');

```

```

o.fillStyle = hex;
o.fillRect(0, 0, w, h);
o.globalCompositeOperation = 'destination-in';
o.drawImage(maskImg, 0, 0, w, h);

ctx.save();
ctx.globalCompositeOperation = 'source-over';
ctx.globalAlpha = brilloPaintAlpha(brilloId);
ctx.drawImage(off, 0, 0);
ctx.restore();
ctx.globalAlpha = 1;

if (brilloId === 'satinado' || brilloId === 'semi_brillo') {
  var gloss = document.createElement('canvas');
  gloss.width = w;
  gloss.height = h;
  var g = gloss.getContext('2d');
  var grad = g.createLinearGradient(w * 0.55, 0, w, h * 0.35);
  grad.addColorStop(0, 'rgba(255,255,255,0)');
  grad.addColorStop(1, brilloId === 'semi_brillo' ? 'rgba(255,255,255,0.25)' : 'rgba(255,255,255,0.14)');
  g.fillStyle = grad;
  g.fillRect(0, 0, w, h);
  g.globalCompositeOperation = 'destination-in';
  g.drawImage(maskImg, 0, 0, w, h);
  ctx.save();
  ctx.globalCompositeOperation = 'source-over';
  ctx.globalAlpha = 0.35;
  ctx.drawImage(gloss, 0, 0);
  ctx.restore();
}
}

function applyPolygonTint(ctx, w, h, polygon, hex, brilloId) {
  ctx.save();
  traceWallPolygon(ctx, polygon, w, h);
  ctx.clip();
  ctx.globalCompositeOperation = 'color';
  ctx.globalAlpha = brilloPaintAlpha(brilloId);
  ctx.fillStyle = hex;
  ctx.fillRect(0, 0, w, h);
  ctx.restore();
}

function drawSceneCanvas() {
  if (!els.sceneCanvas || !els.scene) return;
  var col = colorById(state.color_id);
  var amb = ambienteById(state.ambiente_id);
  var sceneId = sceneForAmbiente(amb);
  var photoUrl = (amb && amb.photo_url) || sceneUrls[sceneId] || sceneUrls.interior || '';
  var maskUrl = (amb && amb.mask_url) || '';
  if (!photoUrl) return;

  var token = ++sceneDrawToken;
  var polygon = wallPolygonFor(amb);
  var hex = (col && col.hex) || '#ECEF1';
  var brilloId = state.brillo_id || 'mate';

  var loads = [loadSceneImage(photoUrl)];
  if (maskUrl) loads.push(loadSceneImage(maskUrl));

  Promise.all(loads).then(function (parts) {
    if (token !== sceneDrawToken) return;
    var img = parts[0];
    var maskImg = parts[1] || null;
    var wrap = els.sceneCanvas.parentElement;
    var cssW = Math.max(280, (wrap && wrap.clientWidth) || 640);
    var cssH = Math.round(cssW * (img.height / img.width));
    var dpr = window.devicePixelRatio || 1;

```

```

    els.sceneCanvas.width = Math.round(cssW * dpr);
    els.sceneCanvas.height = Math.round(cssH * dpr);
    els.sceneCanvas.style.width = cssW + 'px';
    els.sceneCanvas.style.height = cssH + 'px';

    var ctx = els.sceneCanvas.getContext('2d');
    ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
    ctx.clearRect(0, 0, cssW, cssH);
    ctx.drawImage(img, 0, 0, cssW, cssH);

    if (maskImg && maskImg.naturalWidth) {
        applyMaskTint(ctx, cssW, cssH, hex, brilloId, maskImg);
    }
}).catch(function () {
    if (token !== sceneDrawToken || !els.sceneCanvas) return;
    var ctx = els.sceneCanvas.getContext('2d');
    var w = els.sceneCanvas.clientWidth || 640;
    var h = els.sceneCanvas.clientHeight || 360;
    ctx.clearRect(0, 0, w, h);
    ctx.fillStyle = '#eceff1';
    ctx.fillRect(0, 0, w, h);
});
}

function scheduleSceneRedraw() {
    if (sceneResizeTimer) clearTimeout(sceneResizeTimer);
    sceneResizeTimer = setTimeout(drawSceneCanvas, 80);
}

function updateScene() {
    var col = colorById(state.color_id);
    var amb = ambienteById(state.ambiente_id);
    var brillo = brilloById(state.brillo_id);
    var sceneId = sceneForAmbiente(amb);

    if (els.scene) {
        els.scene.setAttribute('data-scene', sceneId);
        if (amb && amb.id) {
            els.scene.setAttribute('data-ambiente', amb.id);
        }
        els.scene.classList.add('fc-scene--canvas');
    }
    drawSceneCanvas();
    if (col) {
        document.documentElement.style.setProperty('--fc-hex', col.hex);
    }
    if (els.sceneSwatch && col) {
        els.sceneSwatch.style.backgroundColor = col.hex;
    }
    if (els.sceneLabel) {
        var parts = [];
        if (amb) parts.push(amb.nombre);
        if (col) parts.push(col.nombre);
        if (brillo) parts.push(brillo.nombre);
        els.sceneLabel.textContent = parts.join(' · ') || 'Vista previa';
    }
}

function lizTipForStep() {
    var amb = ambienteById(state.ambiente_id);
    var brillo = brilloById(state.brillo_id);
    var col = colorById(state.color_id);
    var m2 = parseFloat(state.m2) || 12;

    if (state.step === 1 && amb) {
        if (amb.id === 'bano') {
            return 'En baños conviene satinado o semi brillo: resisten humedad y son fáciles de limpiar.';
        }
    }
}

```



```

    if (amb.id === 'fachada') {
      return 'Para fachada use pintura exterior con filtro UV. Le mostraremos opciones con stock en tienda.';
    }
    if (amb.id === 'dormitorio') {
      return 'En dormitorios, tonos suaves y acabado mate ayudan a un ambiente más relajado.';
    }
    return amb.tip || 'Elija el ambiente y verá una vista previa con el color que seleccione.';
  }
}

if (state.step === 2) {
  if (amb && amb.id === 'bano' && state.brillo_id === 'mate') {
    return 'Para baño, el mate puede marcar manchas de humedad. ¿Probamos satinado? Resiste mejor el vapor.';
  }
  if (amb && amb.id === 'cocina' && state.brillo_id === 'mate') {
    return 'En cocina el satinado facilita limpiar grasa y salpicaduras sin perder elegancia.';
  }
  if (col && col.familia === 'neutro' && amb && amb.id === 'living') {
    return 'Los grises y neutros amplían visualmente el living. Combine con blanco en cielo.';
  }
  if (brillo) {
    return brillo.ideal + '. Puede cambiar el brillo y ver cómo afecta la vista previa.';
  }
}

if (state.step === 3) {
  if (m2 <= 10) {
    return 'Para un dormitorio pequeño (~10 m² de muro) suele alcanzar 1 galón a 2 manos.';
  }
  if (m2 >= 25) {
    return 'Superficies amplias conviene calcular con margen. Incluimos redondeo a cuartos de galón.';
  }
  var c = calcLocalCantidad();
  return 'Con ' + m2 + ' m² estimamos ' + c.galones_sugeridos_fmt + ' galones a 2 manos. Ajuste si incluye cielo.';
}

return '';
}

function renderLizTipLocal() {
  /* Tips contextuales: usar botón flotante Liz (no barra en wizard). */
}

function fetchLizTip() {
  /* deshabilitado en wizard – evita barra confusa; Liz sigue en esquina. */
}

function renderLizTip() {}

function syncPaletaAmbiente() {
  var fams = activeFamilias();
  if (!fams.length) return;
  var ids = fams.map(function (f) { return f.id; });
  if (ids.indexOf(state.familia_id) < 0) {
    state.familia_id = fams[0].id;
  }
  var fam = fams.find(function (f) { return f.id === state.familia_id; });
  var cols = (fam && fam.colores) || [];
  if (!colorById(state.color_id) && cols[0]) {
    state.color_id = cols[0].id;
  }
}

function updateCartillaCount() {
  if (!els.cartillaCount) return;
  var n = allActiveColores().length;
  if (!n && initial.sin_stock) {

```

```

        els.cartillaCount.textContent = '0 – consulte en mostrador';
        return;
    }
    els.cartillaCount.textContent = String(n);
}

function selectAmbiente(id, advance) {
    state.ambiente_id = id;
    sceneImageCache = {};
    sceneDrawToken++;
    syncPaletaAmbiente();
    renderAmbientes();
    renderFamilias();
    renderPaleta();
    updateScene();
    pulseScene();
    renderLizTip();
    if (state.step === 4) fetchCotizacion();
    if (advance && state.step === 1) {
        setStep(2);
    }
}

function renderAmbientes() {
    if (!els.ambientes) return;
    els.ambientes.innerHTML = (initial.ambientes || []).map(function (a) {
        var sel = a.id === state.ambiente_id ? ' is-selected' : '';
        var hero = a.grid_class ? ' ' + a.grid_class : '';
        var photo = a.photo_url ? ' style="background-image:url(\'' + a.photo_url + '\')"' : '';
        return (
            '<button type="button" class="fc-amb-card' + hero + sel + '" data-ambiente="' + a.id + '" + photo + '>'
+
            '<span class="fc-amb-card__label">' + a.nombre + '</span>' +
            '<span class="fc-amb-card__shade"></span></button>'
        );
    }).join('');
    els.ambientes.querySelectorAll('[data-ambiente]').forEach(function (btn) {
        btn.addEventListener('click', function () {
            selectAmbiente(btn.getAttribute('data-ambiente'), true);
        });
    });
}

function renderFamilias() {
    if (!els.familias) return;
    var fams = activeFamilias();
    els.familias.innerHTML = fams.map(function (f) {
        var active = f.id === state.familia_id ? ' is-active' : '';
        return '<button type="button" class="fc-pill' + active + '" data-familia="' + f.id + '">' + f.nombre +
    '</button>';
    }).join('');
    els.familias.querySelectorAll('[data-familia]').forEach(function (btn) {
        btn.addEventListener('click', function () {
            state.familia_id = btn.getAttribute('data-familia');
            var fam = activeFamilias().find(function (f) { return f.id === state.familia_id; });
            if (fam && fam.colores && fam.colores[0]) {
                state.color_id = fam.colores[0].id;
            }
            renderFamilias();
            renderPaleta();
            updateScene();
            pulseScene();
            renderLizTip();
            if (state.step === 4) fetchCotizacion();
        });
    });
}

```

```

function renderPaleta() {
  if (!els.paleta) return;
  var fam = activeFamilias().find(function (f) { return f.id === state.familia_id; });
  var cols = (fam && fam.colores) || allActiveColores();
  if (!state.color_id && cols[0]) state.color_id = cols[0].id;
  els.paleta.innerHTML = cols.map(function (c) {
    var sel = c.id === state.color_id ? ' is-selected' : '';
    return (
      '<button type="button" class="fc-swatch' + sel + '" data-color="' + c.id + '" title="' +
      (c.nombre_completo || c.nombre) + '">' +
      '<span style="background:' + c.hex + '"></span><em>' + c.nombre + '</em>' +
      (stockMode && c.stock_tienda ? '<small class="fc-swatch-stock">' + c.stock_tienda + ' en tienda</small>'
      : '') +
      '<small class="fc-swatch-code">' + (c.codigo || '') + '</small></button>'
    );
  }).join('');
  els.paleta.querySelectorAll('[data-color]').forEach(function (btn) {
    btn.addEventListener('click', function () {
      state.color_id = btn.getAttribute('data-color');
      var col = colorById(state.color_id);
      if (stockMode && col && col.producto_id) {
        state.producto_id = col.producto_id;
      }
      renderPaleta();
      updateScene();
      pulseScene();
      renderLizTip();
      if (state.step === 4) fetchCotizacion();
    });
  });
}

function renderBrillos() {
  if (!els.brillos) return;
  els.brillos.innerHTML = (initial.brillos || []).map(function (b) {
    var sel = b.id === state.brillo_id ? ' is-selected' : '';
    return (
      '<button type="button" class="fc-card-opt' + sel + '" data-brillo="' + b.id + '">' +
      '<strong>' + b.nombre + '</strong>' +
      '<small>' + b.desc + '</small>' +
      '<small class="d-block mt-1">' + b.ideal + '</small></button>'
    );
  }).join('');
  els.brillos.querySelectorAll('[data-brillo]').forEach(function (btn) {
    btn.addEventListener('click', function () {
      state.brillo_id = btn.getAttribute('data-brillo');
      renderBrillos();
      updateScene();
      renderLizTip();
      if (state.step === 4) fetchCotizacion();
    });
  });
}

function renderCantidadResumen() {
  if (!els.cantidadResumen) return;
  var c = calcLocalCantidad();
  els.cantidadResumen.innerHTML =
    '<strong>' + c.galones_sugeridos_fmt + ' galones</strong> estimados para ' +
    c.m2 + ' m2 · ' + c.manos + ' manos (rendimiento referencial).';
}

function renderCalidades() {
  if (!els.calidades) return;
  if (stockMode) {
    els.calidades.innerHTML = '';
    els.calidades.classList.add('d-none');
    return;
  }

```

```

    }
    els.calidades.classList.remove('d-none');
    els.calidades.innerHTML = (initial.calidades || []).map(function (q) {
        var sel = q.id === state.calidad_id ? ' is-selected' : '';
        return (
            '<button type="button" class="fc-calidad-card' + sel + '" data-calidad="' + q.id + '">' +
            (q.badge ? '<span class="fc-calidad-badge">' + q.badge + '</span>' : '') +
            '<strong>' + q.nombre + '</strong></button>'
        );
    }).join('');
    els.calidades.querySelectorAll('[data-calidad]').forEach(function (btn) {
        btn.addEventListener('click', function () {
            state.calidad_id = btn.getAttribute('data-calidad');
            state.producto_id = null;
            renderCalidades();
            fetchCotizacion();
        });
    });
}

```

```

function buildCartLines(data) {
    var lines = [];
    var p = data.producto;
    var tot = data.totales || {};
    var galones = tot.galones || 1;
    if (p) {
        lines.push({
            producto_id: p.producto_id,
            nombre: p.nombre,
            referencia: p.referencia || '',
            precio: p.precio,
            precio_fmt: p.precio_fmt,
            imagen_url: p.imagen_url || '',
            disponible: p.disponible,
            stock_tienda: p.stock_tienda,
            cantidad: galones,
        });
    }
    (data.complementos || []).forEach(function (c) {
        if (!c.disponible) return;
        lines.push({
            producto_id: c.producto_id,
            nombre: c.nombre,
            referencia: '',
            precio: c.precio || 0,
            precio_fmt: c.precio_fmt,
            imagen_url: '',
            disponible: c.disponible,
            stock_tienda: c.stock_tienda,
            cantidad: 1,
        });
    });
    return lines;
}

```

```

function pedirEnCaja(data) {
    var lines = buildCartLines(data);
    if (!lines.length) return;
    if (window.tiendaCarritoReplace) {
        window.tiendaCarritoReplace(lines);
    } else if (window.tiendaCarritoAdd) {
        lines.forEach(function (ln) {
            window.tiendaCarritoAdd(ln, ln.cantidad || 1);
        });
    }
    if (window.tiendaGenerarValePedido) {
        window.tiendaGenerarValePedido();
        return;
    }
}

```

```

    }
    if (window.__tiendaSetCartOpen) {
        window.__tiendaSetCartOpen(true);
    }
}

function renderResultado(data) {
    if (!els.resultado) return;
    if (!data || !data.ok) {
        if (els.proyectoTotal) {
            els.proyectoTotal.classList.add('d-none');
            els.proyectoTotal.innerHTML = '';
        }
        if (els.complementos) els.complementos.innerHTML = '';
        if (els.acciones) els.acciones.innerHTML = '';
        els.resultado.innerHTML = '<p class="text-muted mb-0">No pudimos cotizar. Intente otra calidad.</p>';
        return;
    }
}

state.cotizacion = data;
var p = data.producto;
var tinte = data.tinte || {};
var tot = data.totales || {};

if (p) state.producto_id = p.producto_id;

var imgBlock = p && p.imagen_url
    ? ''
    : '<div class="fc-hero-placeholder"><i class="fas fa-fill-drip fa-2x text-muted"></i></div>';

var stockCls = p && p.disponible ? 'fc-stock-ok' : 'fc-stock-no';
var stockTxt = p && p.disponible ? 'Disponible en tienda ahora' : 'Consultar stock en mostrador';

var heroHtml =
    '<div class="fc-proyecto-hero">' +
    '<div class="fc-proyecto-hero__media">' + imgBlock +
    (tinte.hex ? '<span class="fc-proyecto-hero__color" style="background:' + tinte.hex + '"></span>' : '') +
    '</div>' +
    '<div class="fc-proyecto-hero__copy">' +
    '<div class="fc-proyecto-hero__kicker">Su proyecto</div>' +
    '<h3>' + data.titulo + '</h3>' +
    '<p class="fc-proyecto-hero__meta">' + data.resumen + '</p>';

if (tinte.codigo) {
    heroHtml +=
        '<div class="fc-proyecto-hero__tinte">' +
        '<i class="fas fa-box"></i>' + (data.modos === 'stock_erp' || tinte.modos === 'stock_erp' ? 'En stock . '
: 'Tinte ') +
        tinte.codigo + (tinte.marca ? ' . ' + tinte.marca : '') +
        '</div>';
}

if (p) {
    heroHtml +=
        '<div class="fw-semibold fs-5 mb-1">' + p.nombre + '</div>' +
        '<div class="mb-1">' + p.precio_fmt + ' / gal . ref. ' + (p.referencia || '-') + '</div>' +
        '<div class="" + stockCls + "">' + stockTxt + '</div>';
} else {
    heroHtml += '<p class="text-muted mb-0">No hay pinturas en catálogo para este tier. Consulte en
mostrador.</p>';
}

heroHtml += '</div></div>';
els.resultado.innerHTML = heroHtml;

if (els.proyectoTotal && tot.total_proyecto_fmt) {
    els.proyectoTotal.classList.remove('d-none');
    var compRow = tot.subtotal_complementos > 0

```

```

        ? '<div class="fc-proyecto-total__row"><span>Complementos</span><span>' + tot.subtotal_complementos_fmt
+ '</span></div>'
        : '';
        els.proyectoTotal.innerHTML =
            '<div class="fc-proyecto-total__row"><span>Pintura (' + (tot.galones || 1) + ' gal)</span><span>' +
tot.subtotal_pintura_fmt + '</span></div>' +
            compRow +
            '<div class="fc-proyecto-total__grand"><span>Total referencial proyecto</span><em>' +
tot.total_proyecto_fmt + '</em></div>' +
            '<p class="small mb-0 mt-2 opacity-75">' +
            (stockMode ? 'Stock sujeto a venta en piso. Unidades según formato del envase.' : 'Incluye tinte en
mostrador. Cobro final en caja.') +
            '</p>';
        } else if (els.proyectoTotal) {
            els.proyectoTotal.classList.add('d-none');
        }

        if (els.complementos) {
            var comps = data.complementos || [];
            if (!comps.length) {
                els.complementos.innerHTML = '';
            } else {
                els.complementos.innerHTML =
                    '<h3 class="h6 fw-bold mb-2">Complementos sugeridos</h3>' +
                    '<div class="fc-comp-chips">' +
                    comps.map(function (c) {
                        var cls = c.disponible ? '' : ' is-unavailable';
                        var st = c.disponible ? ' · en tienda' : ' · consultar';
                        return (
                            '<span class="fc-comp-chip' + cls + '">' +
                            '<i class="fas fa-check-circle"></i>' +
                            '<span>' + c.nombre + st + ' · ' + c.precio_fmt + '</span></span>'
                        );
                    }).join('') +
                    '</div>';
            }
        }

        if (els.acciones) {
            var cajaBtn = '';
            if (cfg.modo_caja && p) {
                cajaBtn =
                    '<button type="button" class="fc-btn fc-btn--caja" id="fcPedirCaja">' +
                    '<i class="fas fa-cash-register me-1"></i> Pedir en caja</button>';
            }

            var wa = '';
            if (data.mensaje_whatsapp) {
                var waNum = (window.tiendaCarritoConfig && window.tiendaCarritoConfig.whatsapp) || '';
                if (waNum) {
                    wa =
                        '<a class="fc-btn fc-btn--wa" target="_blank" rel="noopener" href="https://wa.me/' +
                        waNum + '?text=' + encodeURIComponent(data.mensaje_whatsapp) + '">' +
                        '<i class="fab fa-whatsapp me-1"></i> WhatsApp</a>';
                }
            }

            var lizBtn = data.liz_prompt
            ? '<button type="button" class="fc-btn fc-btn--ghost" id="fcAskLiz">Preguntar a Liz</button>'
            : '';

            var cartBtn = p
            ? '<button type="button" class="fc-btn fc-btn--primary" id="fcAddCart">Agregar al carrito</button>'
            : '';

            els.acciones.className = 'fc-acciones fc-acciones--hero';
            els.acciones.innerHTML = cajaBtn + cartBtn + lizBtn + wa;

```

```

var pedirBtn = document.getElementById('fcPedirCaja');
if (pedirBtn) {
  pedirBtn.addEventListener('click', function () {
    pedirEnCaja(data);
  });
}

var addBtn = document.getElementById('fcAddCart');
if (addBtn && p && window.tiendaCarritoAdd) {
  addBtn.addEventListener('click', function () {
    var galones = (tot.galones || 1);
    window.tiendaCarritoAdd({
      producto_id: p.producto_id,
      nombre: p.nombre,
      referencia: p.referencia || '',
      precio: p.precio,
      precio_fmt: p.precio_fmt,
      imagen_url: p.imagen_url || '',
      disponible: p.disponible,
      stock_tienda: p.stock_tienda,
    }, galones);
  });
}

var liz = document.getElementById('fcAskLiz');
if (liz && data.liz_prompt) {
  liz.addEventListener('click', function () {
    var toggle = document.getElementById('tiendaAssistantToggle');
    var input = document.getElementById('tiendaAssistantInput');
    if (toggle) toggle.click();
    if (input) {
      input.value = data.liz_prompt;
      input.focus();
    }
  });
}

}

function fetchCotizacion() {
  if (!cfg.cotizar_url) return;
  els.resultado.innerHTML = '<p class="text-muted mb-0"><i class="fas fa-spinner fa-spin me-1"></i> Calculando proyecto...</p>';
  if (els.proyectoTotal) els.proyectoTotal.classList.add('d-none');
  if (els.complementos) els.complementos.innerHTML = '';
  if (els.acciones) els.acciones.innerHTML = '';

  fetch(cfg.cotizar_url, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      ambiente_id: state.ambiente_id,
      color_id: state.color_id,
      brillo_id: state.brillo_id,
      m2: state.m2,
      calidad_id: state.calidad_id,
      producto_id: state.producto_id,
    }),
  })
  .then(function (r) { return r.json(); })
  .then(renderResultado)
  .catch(function () {
    renderResultado(null);
  });
}

function setStep(n) {
  state.step = Math.max(1, Math.min(4, n));
}

```

```

    if (els.main) {
        els.main.setAttribute('data-step', String(state.step));
    }
    if (els.pick) {
        els.pick.classList.toggle('is-active', state.step === 1);
        if (state.step === 1) {
            els.pick.removeAttribute('hidden');
        } else {
            els.pick.setAttribute('hidden', '');
        }
    }
    if (els.workspace) {
        var showWs = state.step >= 2;
        els.workspace.classList.toggle('is-active', showWs);
        if (showWs) {
            els.workspace.removeAttribute('hidden');
        } else {
            els.workspace.setAttribute('hidden', '');
        }
    }
    document.querySelectorAll('.fc-step').forEach(function (sec) {
        sec.classList.toggle('is-active', parseInt(sec.getAttribute('data-step'), 10) === state.step);
    });
    if (els.stepsNav) {
        els.stepsNav.querySelectorAll('li').forEach(function (li) {
            var s = parseInt(li.getAttribute('data-step'), 10);
            li.classList.toggle('is-active', s === state.step);
            li.classList.toggle('is-done', s < state.step);
        });
    }
    if (els.progressFill) {
        els.progressFill.style.width = (state.step * 25) + '%';
    }
    if (els.prev) els.prev.disabled = state.step <= 1;
    if (els.next) {
        els.next.textContent = state.step >= 4 ? 'Recalcular' : 'Siguiente';
    }
    renderLizTip();
    if (state.step === 4) {
        fetchCotizacion();
    }
    if (state.step >= 2) {
        scheduleSceneRedraw();
    }
}

if (els.m2) {
    els.m2.addEventListener('input', function () {
        state.m2 = parseFloat(els.m2.value) || 1;
        renderCantidadResumen();
        renderLizTip();
    });
    document.querySelectorAll('[data-m2]').forEach(function (btn) {
        btn.addEventListener('click', function () {
            state.m2 = parseFloat(btn.getAttribute('data-m2')) || 12;
            els.m2.value = state.m2;
            renderCantidadResumen();
            renderLizTip();
        });
    });
}

if (els.changeAmbiente) {
    els.changeAmbiente.addEventListener('click', function () {
        setStep(1);
    });
}

```



```

    if (els.prev) {
      els.prev.addEventListener('click', function () {
        setStep(state.step - 1);
      });
    }
    if (els.next) {
      els.next.addEventListener('click', function () {
        if (state.step >= 4) {
          fetchCotizacion();
          return;
        }
        if (state.step === 3) renderCantidadResumen();
        setStep(state.step + 1);
      });
    }
  }

  syncPaletaAmbiente();
  if (stockMode && state.color_id) {
    var c0 = colorById(state.color_id);
    if (c0 && c0.producto_id) state.producto_id = c0.producto_id;
  }
  renderAmbientes();
  renderFamilias();
  renderPaleta();
  renderBrillos();
  renderCantidadResumen();
  renderCalidades();
  updateCartillaCount();
  updateScene();
  setStep(1);

  window.addEventListener('resize', scheduleSceneRedraw);
})();

```

static/css/fabrica-color.css

```

/* Fábrica de Color – vitrina LhexIA */
.fabrica-color-page {
  --fc-accent: #c62828;
  --fc-accent-soft: rgba(198, 40, 40, 0.12);
  --fc-wall: #ecefff;
  --fc-hex: #ecefff;
}

.fc-main {
  max-width: 1280px;
  margin: 0 auto;
  padding: 1rem 1.25rem 3rem;
}

.fc-top {
  margin-bottom: 0.65rem;
}

.fc-top h1 {
  font-size: clamp(1.35rem, 3vw, 1.75rem);
  font-weight: 800;
  margin-bottom: 0;
  letter-spacing: -0.02em;
}

.fabrica-color-page .fc-kicker {
  display: none;
}

```

```
.fc-link-back {
  font-size: 0.85rem;
  color: #546e7a;
  text-decoration: none;
}

.fc-link-back:hover {
  color: var(--fc-accent);
}

/* — Paso 1: grid de ambientes — */
.fc-pick {
  display: none;
}

.fc-pick.is-active {
  display: block;
}

.fc-pick__title {
  text-align: center;
  font-size: clamp(1.35rem, 3vw, 1.85rem);
  font-weight: 700;
  margin-bottom: 0.35rem;
  color: #263238;
}

.fc-pick__lead {
  text-align: center;
  color: #78909c;
  font-size: 0.95rem;
  margin-bottom: 1.25rem;
}

.fc-amb-grid {
  display: grid;
  grid-template-columns: 1.15fr 1fr 1fr;
  grid-template-rows: minmax(200px, 26vh) minmax(200px, 26vh) minmax(160px, 20vh);
  gap: 12px;
}

.fc-amb-card[data-ambiente="comedor"] {
  grid-column: 1;
  grid-row: 1 / 3;
}

.fc-amb-card[data-ambiente="cocina"] {
  grid-column: 2;
  grid-row: 1;
}

.fc-amb-card[data-ambiente="dormitorio"] {
  grid-column: 3;
  grid-row: 1;
}

.fc-amb-card[data-ambiente="bano"] {
  grid-column: 2;
  grid-row: 2;
}

.fc-amb-card[data-ambiente="living"] {
  grid-column: 3;
  grid-row: 2;
}

.fc-amb-card[data-ambiente="fachada"] {
```

```

    grid-column: 2 / 4;
    grid-row: 3;
}

@media (max-width: 900px) {
    .fc-amb-grid {
        grid-template-columns: 1fr 1fr;
        grid-template-rows: auto;
    }

    .fc-amb-card[data-ambiente] {
        grid-column: auto !important;
        grid-row: auto !important;
    }

    .fc-amb-card[data-ambiente="comedor"] {
        grid-column: 1 / -1;
        min-height: 260px;
    }

    .fc-amb-card[data-ambiente="fachada"] {
        grid-column: 1 / -1;
    }
}

@media (max-width: 520px) {
    .fc-amb-grid {
        grid-template-columns: 1fr;
    }
}

.fc-amb-card {
    position: relative;
    border: none;
    border-radius: 14px;
    overflow: hidden;
    padding: 0;
    cursor: pointer;
    background: #263238 center/cover no-repeat;
    min-height: 180px;
    box-shadow: 0 8px 28px rgba(38, 50, 56, 0.14);
    transition: transform 0.25s ease, box-shadow 0.25s ease;
}

.fc-amb-card:hover {
    transform: translateY(-3px);
    box-shadow: 0 14px 36px rgba(38, 50, 56, 0.2);
}

.fc-amb-card.is-selected {
    box-shadow: 0 0 0 3px var(--fc-accent), 0 14px 36px rgba(198, 40, 40, 0.25);
}

.fc-amb-card--hero {
    grid-row: 1 / 3;
    min-height: 100%;
}

.fc-amb-card__label {
    position: absolute;
    top: 12px;
    left: 12px;
    background: rgba(0, 0, 0, 0.72);
    color: #fff;
    font-size: 0.82rem;
    font-weight: 700;
    padding: 0.35rem 0.75rem;
    border-radius: 6px;
}

```

```

    letter-spacing: 0.02em;
}

.fc-amb-card__shade {
    position: absolute;
    inset: 0;
    background: linear-gradient(180deg, transparent 40%, rgba(0, 0, 0, 0.35) 100%);
    pointer-events: none;
}

/* — Workspace pasos 2-4 — */
.fc-workspace {
    display: none;
    grid-template-columns: minmax(0, 1.15fr) minmax(300px, 420px);
    gap: 1.25rem;
    align-items: start;
}

.fc-workspace.is-active {
    display: grid;
}

@media (max-width: 960px) {
    .fc-workspace.is-active {
        grid-template-columns: 1fr;
    }
}

.fc-workspace__preview {
    position: sticky;
    top: 1rem;
}

.fc-workspace__preview .fc-scene {
    min-height: 0;
    border-radius: 18px;
    background: #eceff1;
}

.fc-scene--canvas {
    position: relative;
    line-height: 0;
}

.fc-scene__canvas {
    display: block;
    width: 100%;
    height: auto;
    border-radius: 18px;
    vertical-align: top;
}

.fc-scene__disclaimer {
    position: absolute;
    bottom: 10px;
    right: 12px;
    margin: 0;
    font-size: 0.68rem;
    color: rgba(255, 255, 255, 0.92);
    background: rgba(0, 0, 0, 0.45);
    padding: 0.2rem 0.5rem;
    border-radius: 4px;
    line-height: 1.3;
    pointer-events: none;
}

.fc-change-ambiente {
    display: inline-flex;

```

```

    align-items: center;
    margin-top: 0.65rem;
    border: none;
    background: none;
    color: #546e7a;
    font-size: 0.88rem;
    font-weight: 600;
    cursor: pointer;
    padding: 0.25rem 0;
}

.fc-change-ambiente:hover {
    color: var(--fc-accent);
}

.fc-workspace__panel {
    background: #fff;
    border: 1px solid #eceff1;
    border-radius: 18px;
    box-shadow: 0 4px 24px rgba(38, 50, 56, 0.06);
    overflow: hidden;
}

.fc-workspace__panel .fc-wizard {
    padding: 1.25rem 1.35rem 0.5rem;
}

.fc-panel-section {
    margin-bottom: 1.15rem;
}

.fc-panel-heading {
    font-size: 0.78rem;
    font-weight: 700;
    letter-spacing: 0.06em;
    text-transform: uppercase;
    color: #78909c;
    margin-bottom: 0.55rem;
}

.fc-workspace__panel .fc-nav {
    padding: 0.85rem 1.35rem 1.15rem;
    border-top: 1px solid #eceff1;
    margin-top: 0;
}

.fc-hero {
    display: none;
}

.fc-kicker {
    display: inline-block;
    font-size: 0.75rem;
    font-weight: 700;
    letter-spacing: 0.06em;
    text-transform: uppercase;
    color: var(--fc-accent);
    margin-bottom: 0.35rem;
}

.fc-hero h1 {
    font-size: clamp(1.6rem, 4vw, 2.2rem);
    font-weight: 800;
    margin-bottom: 0.5rem;
}

.fc-hero p {
    color: #546e7a;

```

```

    margin-bottom: 0.75rem;
}

.fc-link-back {
    font-size: 0.9rem;
    color: #37474f;
    text-decoration: none;
}

.fc-link-back:hover {
    color: var(--fc-accent);
}

.fc-scene {
    position: relative;
    border-radius: 16px;
    overflow: hidden;
    min-height: 200px;
    background: linear-gradient(180deg, #90caf9 0%, #bbdefb 42%, #cfd8dc 100%);
    box-shadow: inset 0 0 0 1px rgba(0, 0, 0, 0.06), 0 12px 32px rgba(38, 50, 56, 0.12);
    transition: background 0.45s ease;
}

.fc-scene--photo {
    background: #263238;
}

.fc-scene--photo .fc-scene__sky,
.fc-scene--photo .fc-scene__floor,
.fc-scene--photo .fc-scene__window,
.fc-scene--photo .fc-scene__props {
    display: none !important;
}

.fc-scene--photo .fc-scene__wall {
    mix-blend-mode: multiply;
    opacity: 0.5;
    border-radius: 2px;
    box-shadow: none;
    pointer-events: none;
    -webkit-mask-image: linear-gradient(to bottom, transparent 0%, #000 5%, #000 90%, transparent 100%);
    mask-image: linear-gradient(to bottom, transparent 0%, #000 5%, #000 90%, transparent 100%);
}

.fc-scene--photo[data-ambiente="cocina"] .fc-scene__wall {
    opacity: 0.48;
}

.fc-scene--photo[data-ambiente="fachada"] .fc-scene__wall {
    mix-blend-mode: multiply;
    opacity: 0.58;
    border-radius: 4px 4px 0 0;
}

.fc-scene--photo[data-ambiente="bano"] .fc-scene__wall {
    opacity: 0.45;
}

.fc-scene--photo .fc-scene__label {
    background: rgba(0, 0, 0, 0.55);
    backdrop-filter: blur(6px);
}

/* Zona muro por ambiente (refuerzo CSS; JS aplica inset desde wall_zone) */
.fc-scene--photo[data-ambiente="living"] .fc-scene__wall {
    opacity: 0.52;
}

```

```

.fc-scene[data-scene="fachada"] {
  background: linear-gradient(180deg, #81d4fa 0%, #b3e5fc 38%, #a5d6a7 100%);
}

.fc-scene[data-scene="bano"] {
  background: linear-gradient(180deg, #eceff1 0%, #cfd8dc 100%);
}

.fc-scene__sky {
  position: absolute;
  inset: 0 0 55% 0;
  background: linear-gradient(180deg, rgba(255, 255, 255, 0.15) 0%, transparent 100%);
  pointer-events: none;
}

.fc-scene__photo {
  position: absolute;
  inset: 0;
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
  transition: opacity 0.45s ease;
  opacity: 1;
}

.fc-scene__wall {
  position: absolute;
  inset: 0 0 0 0;
  background: var(--fc-hex);
  transition: background-color 0.45s ease, filter 0.35s ease, opacity 0.35s ease;
  mix-blend-mode: multiply;
  opacity: 0.55;
  mask-image: linear-gradient(180deg, #000 0%, #000 72%, transparent 100%);
  -webkit-mask-image: linear-gradient(180deg, #000 0%, #000 72%, transparent 100%);
}

.fc-scene[data-scene="fachada"] .fc-scene__wall {
  left: 18%;
  right: 18%;
  bottom: 22%;
  top: 28%;
  border-radius: 4px 4px 0 0;
  box-shadow: inset 0 -8px 0 rgba(0, 0, 0, 0.08);
}

.fc-scene[data-scene="bano"] .fc-scene__wall {
  inset: 0 0 32% 0;
}

.fc-scene__wall--brillo-mate {
  filter: saturate(1) brightness(0.98);
}

.fc-scene__wall--brillo-satinado {
  filter: saturate(1.05) brightness(1.04);
  box-shadow: inset -20px 0 40px rgba(255, 255, 255, 0.12);
}

.fc-scene__wall--brillo-semi_brillo {
  filter: saturate(1.08) brightness(1.08);
  box-shadow: inset -30px 0 50px rgba(255, 255, 255, 0.22);
}

.fc-scene__floor {
  position: absolute;
  left: 0;
  right: 0;
  bottom: 0;
}

```

```

    height: 28%;
    background: linear-gradient(180deg, #8d6e63 0%, #6d4c41 100%);
}

.fc-scene[data-scene="bano"] .fc-scene__floor {
    height: 32%;
    background: linear-gradient(180deg, #b0bec5 0%, #78909c 100%);
}

.fc-scene[data-scene="cocina"] .fc-scene__floor {
    background: linear-gradient(180deg, #bcaaa4 0%, #8d6e63 100%);
}

.fc-scene[data-scene="fachada"] .fc-scene__floor {
    height: 22%;
    background: linear-gradient(180deg, #66bb6a 0%, #43a047 100%);
}

.fc-scene__window {
    position: absolute;
    top: 14%;
    right: 10%;
    width: 22%;
    height: 18%;
    border: 3px solid rgba(255, 255, 255, 0.7);
    border-radius: 4px;
    background: linear-gradient(135deg, rgba(255, 255, 255, 0.5), rgba(144, 202, 249, 0.35));
    box-shadow: inset 0 0 12px rgba(255, 255, 255, 0.4);
    opacity: 0;
    transition: opacity 0.3s ease;
}

.fc-scene[data-scene="interior"] .fc-scene__window {
    opacity: 1;
}

.fc-scene__props {
    position: absolute;
    inset: 0;
    pointer-events: none;
    display: none;
}

.fc-scene[data-scene="interior"] .fc-scene__props--interior,
.fc-scene[data-scene="bano"] .fc-scene__props--bano,
.fc-scene[data-scene="cocina"] .fc-scene__props--cocina,
.fc-scene[data-scene="fachada"] .fc-scene__props--fachada {
    display: block;
}

.fc-prop {
    position: absolute;
    transition: opacity 0.35s ease;
}

.fc-prop--sofa {
    left: 10%;
    bottom: 30%;
    width: 36%;
    height: 14%;
    border-radius: 10px 10px 4px 4px;
    background: linear-gradient(180deg, #546e7a 0%, #37474f 100%);
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.2);
}

.fc-prop--table {
    right: 12%;
    bottom: 28%;
}

```



```

width: 16%;
height: 20%;
border-radius: 6px;
background: linear-gradient(180deg, #795548 0%, #5d4037 100%);
}

.fc-prop--lamp {
  right: 28%;
  bottom: 48%;
  width: 4%;
  height: 14%;
  border-radius: 999px 999px 2px 2px;
  background: #ffb74d;
}

.fc-prop--mirror {
  left: 8%;
  top: 12%;
  width: 18%;
  height: 22%;
  border: 3px solid #cfd8dc;
  border-radius: 8px;
  background: linear-gradient(135deg, rgba(255, 255, 255, 0.6), rgba(176, 190, 197, 0.4));
}

.fc-prop--sink {
  right: 10%;
  bottom: 36%;
  width: 28%;
  height: 10%;
  border-radius: 6px;
  background: #e0e0e0;
  box-shadow: inset 0 2px 4px rgba(0, 0, 0, 0.08);
}

.fc-prop--tiles {
  left: 0;
  right: 0;
  bottom: 32%;
  height: 8%;
  background: repeating-linear-gradient(
    90deg,
    #cfd8dc 0,
    #cfd8dc 12px,
    #b0bec5 12px,
    #b0bec5 24px
  );
  opacity: 0.5;
}

.fc-prop--cabinet {
  left: 0;
  right: 0;
  top: 0;
  height: 22%;
  background: linear-gradient(180deg, #78909c 0%, #546e7a 100%);
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.15);
}

.fc-prop--counter {
  left: 0;
  right: 0;
  bottom: 28%;
  height: 6%;
  background: #bdbdbd;
}

.fc-prop--hood {

```

```

    left: 38%;
    top: 18%;
    width: 24%;
    height: 8%;
    border-radius: 4px;
    background: #455a64;
}

.fc-prop--roof {
    left: 12%;
    right: 12%;
    top: 18%;
    height: 12%;
    background: #5d4037;
    clip-path: polygon(50% 0%, 100% 100%, 0% 100%);
}

.fc-prop--door {
    left: 42%;
    bottom: 22%;
    width: 16%;
    height: 18%;
    background: #6d4c41;
    border-radius: 4px 4px 0 0;
}

.fc-prop--lawn {
    left: 0;
    right: 0;
    bottom: 0;
    height: 22%;
    background: transparent;
}

.fc-prop--path {
    left: 44%;
    bottom: 22%;
    width: 12%;
    height: 22%;
    background: linear-gradient(180deg, #bcaaa4 0%, #a1887f 100%);
    opacity: 0.85;
}

.fc-scene__swatch {
    position: absolute;
    top: 10px;
    right: 10px;
    width: 2rem;
    height: 2rem;
    border-radius: 50%;
    border: 2px solid rgba(255, 255, 255, 0.9);
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.2);
    transition: background-color 0.45s ease;
}

.fc-scene__label {
    position: absolute;
    left: 10px;
    bottom: 10px;
    font-size: 0.72rem;
    font-weight: 600;
    color: rgba(255, 255, 255, 0.95);
    background: rgba(0, 0, 0, 0.42);
    padding: 0.25rem 0.6rem;
    border-radius: 999px;
    max-width: 85%;
    white-space: nowrap;
    overflow: hidden;
}

```

```

    text-overflow: ellipsis;
}

.fc-scene.is-updating .fc-scene__canvas {
  animation: fc-wall-pulse 0.5s ease;
}

@keyframes fc-wall-pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.008); }
  100% { transform: scale(1); }
}

.fc-progress {
  margin-bottom: 1.25rem;
}

.fc-progress__bar {
  height: 4px;
  background: #eceff1;
  border-radius: 999px;
  overflow: hidden;
  margin-bottom: 0.75rem;
}

.fc-progress__bar span {
  display: block;
  height: 100%;
  width: 25%;
  background: var(--fc-accent);
  transition: width 0.3s ease;
}

.fc-steps-nav {
  list-style: none;
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem 1rem;
  padding: 0;
  margin: 0;
  font-size: 0.82rem;
  color: #78909c;
}

.fc-steps-nav li {
  display: flex;
  align-items: center;
  gap: 0.35rem;
}

.fc-steps-nav li span {
  width: 1.35rem;
  height: 1.35rem;
  border-radius: 50%;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  font-size: 0.72rem;
  font-weight: 700;
  background: #eceff1;
  color: #546e7a;
}

.fc-steps-nav li.is-active {
  color: #263238;
  font-weight: 600;
}

```

```

.fc-steps-nav li.is-active span,
.fc-steps-nav li.is-done span {
  background: var(--fc-accent);
  color: #fff;
}

.fc-wizard {
  background: #fff;
  border-radius: 16px;
  padding: 1.25rem;
  box-shadow: 0 8px 28px rgba(38, 50, 56, 0.08);
  min-height: 320px;
}

.fc-step {
  display: none;
}

.fc-step.is-active {
  display: block;
}

.fc-step h2 {
  font-size: 1.25rem;
  font-weight: 700;
  margin-bottom: 0.25rem;
}

.fc-step-lead {
  color: #607d8b;
  font-size: 0.92rem;
  margin-bottom: 1rem;
}

.fc-ambientes {
  display: none;
}

.fc-card-opt {
  border: 2px solid #eceff1;
  border-radius: 12px;
  padding: 0.85rem;
  background: #fafafa;
  cursor: pointer;
  text-align: left;
  transition: border-color 0.2s, box-shadow 0.2s;
}

.fc-card-opt:hover {
  border-color: #cfd8dc;
}

.fc-card-opt.is-selected {
  border-color: var(--fc-accent);
  box-shadow: 0 0 0 3px var(--fc-accent-soft);
  background: #fff;
}

.fc-card-opt i {
  color: var(--fc-accent);
  font-size: 1.2rem;
  margin-bottom: 0.35rem;
}

.fc-card-opt strong {
  display: block;
  font-size: 0.95rem;
}

```

```

.fc-card-opt small {
  color: #78909c;
  font-size: 0.78rem;
}

.fc-familias {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
  margin-bottom: 0.75rem;
}

.fc-pill {
  border: 1px solid #cfd8dc;
  background: #fff;
  border-radius: 999px;
  padding: 0.35rem 0.85rem;
  font-size: 0.82rem;
  cursor: pointer;
}

.fc-pill.is-active {
  border-color: var(--fc-accent);
  background: var(--fc-accent-soft);
  color: #b71c1c;
  font-weight: 600;
}

.fc-paleta {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(72px, 1fr));
  gap: 0.5rem;
  margin-bottom: 1rem;
}

.fc-swatch {
  border: 2px solid transparent;
  border-radius: 10px;
  padding: 0;
  overflow: hidden;
  cursor: pointer;
  background: #fff;
}

.fc-swatch span {
  display: block;
  height: 44px;
}

.fc-swatch em {
  display: block;
  font-style: normal;
  font-size: 0.68rem;
  padding: 0.25rem 0.35rem 0;
  text-align: center;
  color: #455a64;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

.fc-swatch-stock {
  display: block;
  font-size: 0.58rem;
  font-weight: 700;
  color: #2e7d32;
  text-align: center;
}

```

```

}

.fc-swatch-code {
  display: block;
  font-size: 0.6rem;
  color: #90a4ae;
  text-align: center;
  padding-bottom: 0.2rem;
}

.fc-swatch.is-selected {
  border-color: var(--fc-accent);
  box-shadow: 0 0 2px var(--fc-accent-soft);
}

.fc-brillos {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
  gap: 0.65rem;
}

.fc-cantidad-form label {
  font-weight: 600;
  font-size: 0.9rem;
  margin-bottom: 0.35rem;
  display: block;
}

.fc-m2-row {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
  align-items: center;
}

.fc-m2-row input {
  width: 120px;
  border: 1px solid #cfd8dc;
  border-radius: 8px;
  padding: 0.5rem 0.65rem;
  font-size: 1rem;
}

.fc-cantidad-resumen {
  margin-top: 1rem;
  padding: 0.85rem 1rem;
  border-radius: 10px;
  background: #f5f7f8;
  font-size: 0.92rem;
}

.fc-calidades {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 0.65rem;
  margin-bottom: 1rem;
}

@media (max-width: 576px) {
  .fc-calidades {
    grid-template-columns: 1fr;
  }
}

.fc-calidad-card {
  position: relative;
  border: 2px solid #eceff1;
  border-radius: 12px;

```

```

padding: 0.85rem;
cursor: pointer;
text-align: center;
}

.fc-calidad-card.is-selected {
border-color: var(--fc-accent);
}

.fc-calidad-badge {
position: absolute;
top: -8px;
left: 50%;
transform: translateX(-50%);
background: var(--fc-accent);
color: #fff;
font-size: 0.65rem;
font-weight: 700;
padding: 0.15rem 0.5rem;
border-radius: 999px;
white-space: nowrap;
}

.fc-resultado {
border: 1px solid #eceeff1;
border-radius: 12px;
padding: 1rem;
margin-bottom: 1rem;
}

.fc-resultado--hero {
border: none;
padding: 0;
background: transparent;
}

.fc-proyecto-hero {
display: grid;
grid-template-columns: minmax(120px, 160px) 1fr;
gap: 1rem;
padding: 1.25rem;
border-radius: 16px;
background: linear-gradient(135deg, #fff 0%, #fafafa 100%);
border: 1px solid #eceeff1;
box-shadow: 0 8px 24px rgba(38, 50, 56, 0.06);
margin-bottom: 1rem;
}

@media (max-width: 576px) {
.fc-proyecto-hero {
grid-template-columns: 1fr;
}
}

.fc-proyecto-hero__media {
position: relative;
}

.fc-proyecto-hero__media img,
.fc-proyecto-hero__media .fc-hero-placeholder {
width: 100%;
aspect-ratio: 1;
object-fit: contain;
border-radius: 12px;
background: #f5f7f8;
}

.fc-proyecto-hero__color {

```

```

    position: absolute;
    bottom: 8px;
    right: 8px;
    width: 2.25rem;
    height: 2.25rem;
    border-radius: 50%;
    border: 2px solid #fff;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.15);
}

.fc-proyecto-hero__kicker {
    font-size: 0.72rem;
    font-weight: 700;
    letter-spacing: 0.05em;
    text-transform: uppercase;
    color: var(--fc-accent);
    margin-bottom: 0.25rem;
}

.fc-proyecto-hero h3 {
    font-size: 1.15rem;
    font-weight: 800;
    margin-bottom: 0.35rem;
}

.fc-proyecto-hero__meta {
    color: #607d8b;
    font-size: 0.88rem;
    margin-bottom: 0.65rem;
}

.fc-proyecto-hero__tinte {
    display: inline-flex;
    align-items: center;
    gap: 0.35rem;
    font-size: 0.82rem;
    font-weight: 600;
    background: #f5f7f8;
    padding: 0.35rem 0.65rem;
    border-radius: 999px;
    margin-bottom: 0.5rem;
}

.fc-proyecto-total {
    padding: 1rem 1.25rem;
    border-radius: 14px;
    background: linear-gradient(135deg, #263238 0%, #37474f 100%);
    color: #fff;
    margin-bottom: 1rem;
}

.fc-proyecto-total__row {
    display: flex;
    justify-content: space-between;
    gap: 0.75rem;
    font-size: 0.88rem;
    opacity: 0.9;
    margin-bottom: 0.35rem;
}

.fc-proyecto-total__grand {
    display: flex;
    justify-content: space-between;
    align-items: baseline;
    gap: 0.75rem;
    margin-top: 0.65rem;
    padding-top: 0.65rem;
    border-top: 1px solid rgba(255, 255, 255, 0.15);
}

```



```

    font-size: 1.05rem;
    font-weight: 700;
}

.fc-proyecto-total__grand em {
    font-style: normal;
    font-size: 1.35rem;
    color: #ffab91;
}

.fc-comp-chips {
    display: flex;
    flex-wrap: wrap;
    gap: 0.5rem;
    margin-top: 0.75rem;
}

.fc-comp-chip {
    display: inline-flex;
    align-items: center;
    gap: 0.35rem;
    padding: 0.4rem 0.75rem;
    border-radius: 999px;
    background: #f5f7f8;
    border: 1px solid #eceff1;
    font-size: 0.8rem;
}

.fc-comp-chip.is-unavailable {
    opacity: 0.65;
}

.fc-comp-chip i {
    color: var(--fc-accent);
    font-size: 0.75rem;
}

.fc-liz-tip {
    display: flex;
    gap: 0.75rem;
    align-items: flex-start;
    padding: 0.85rem 1rem;
    margin-bottom: 1rem;
    border-radius: 12px;
    background: linear-gradient(135deg, #fce4ec 0%, #f3e5f5 100%);
    border: 1px solid rgba(156, 39, 176, 0.15);
    animation: fc-liz-in 0.35s ease;
}

.fc-liz-tip.d-none {
    display: none !important;
}

@keyframes fc-liz-in {
    from { opacity: 0; transform: translateY(6px); }
    to { opacity: 1; transform: translateY(0); }
}

.fc-liz-tip__avatar {
    width: 2rem;
    height: 2rem;
    border-radius: 50%;
    background: #7b1fa2;
    color: #fff;
    display: flex;
    align-items: center;
    justify-content: center;
    flex-shrink: 0;

```

```

}

.fc-liz-tip__body strong {
  display: block;
  font-size: 0.78rem;
  color: #6a1b9a;
  margin-bottom: 0.15rem;
}

.fc-liz-tip__body p {
  margin: 0;
  font-size: 0.88rem;
  color: #4a148c;
  line-height: 1.4;
}

.fc-btn--caja {
  background: linear-gradient(135deg, #ff6f00 0%, #e65100 100%);
  color: #fff;
  font-size: 1rem;
  padding: 0.7rem 1.35rem;
}

.fc-btn--caja:hover {
  filter: brightness(1.05);
  color: #fff;
}

.fc-acciones--hero {
  margin-top: 0.5rem;
}

@media (min-width: 1024px) {
  .fc-main {
    max-width: 1200px;
    padding: 1.5rem 1.5rem 3rem;
  }

  .fc-hero {
    grid-template-columns: 1fr minmax(280px, 420px);
    min-height: 240px;
  }

  .fc-scene {
    min-height: 240px;
  }

  .fc-step h2 {
    font-size: 1.45rem;
  }

  .fc-wizard {
    padding: 1.5rem 1.75rem;
  }

  .fc-proyecto-hero h3 {
    font-size: 1.35rem;
  }
}

/* Modo TV / kiosk mostrador */
.fc-tv-mode .fc-main {
  max-width: 1400px;
  padding: 1.75rem 2rem 3rem;
}

.fc-tv-mode .fc-top h1 {
  font-size: 2.4rem;
}

```

```

}

.fc-tv-mode .fc-workspace__preview .fc-scene {
  min-height: min(58vh, 560px);
}

.fc-tv-mode .fc-step h2 {
  font-size: 1.65rem;
}

.fc-tv-mode .fc-card-opt,
.fc-tv-mode .fc-swatch em {
  font-size: 0.95rem;
}

.fc-tv-mode .fc-swatch span {
  height: 52px;
}

.fc-tv-mode .fc-nav .fc-btn--primary {
  font-size: 1.1rem;
  padding: 0.75rem 1.75rem;
}

.fc-tv-mode .fc-liz-tip__body p {
  font-size: 1rem;
}

.fc-liz-tip.is-loading .fc-liz-tip__body p {
  opacity: 0.55;
}

.fc-liz-tip__fuente {
  font-size: 0.68rem;
  opacity: 0.65;
  margin-top: 0.2rem;
}

.fc-producto-row {
  display: flex;
  gap: 0.85rem;
  align-items: flex-start;
}

.fc-producto-row img {
  width: 72px;
  height: 72px;
  object-fit: contain;
  border-radius: 8px;
  background: #fafafa;
}

.fc-stock-ok {
  color: #2e7d32;
  font-weight: 600;
  font-size: 0.85rem;
}

.fc-stock-no {
  color: #c62828;
  font-weight: 600;
  font-size: 0.85rem;
}

.fc-complementos h3 {
  font-size: 0.95rem;
  margin-bottom: 0.5rem;
}

```

```

.fc-complementos ul {
  list-style: none;
  padding: 0;
  margin: 0;
}

.fc-complementos li {
  display: flex;
  justify-content: space-between;
  gap: 0.5rem;
  padding: 0.45rem 0;
  border-bottom: 1px solid #eceff1;
  font-size: 0.88rem;
}

.fc-acciones {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
  margin-top: 1rem;
}

.fc-nav {
  display: flex;
  justify-content: space-between;
  margin-top: 1rem;
}

.fc-btn {
  border: none;
  border-radius: 10px;
  padding: 0.55rem 1.1rem;
  font-weight: 600;
  cursor: pointer;
}

.fc-btn:disabled {
  opacity: 0.45;
  cursor: not-allowed;
}

.fc-btn--primary {
  background: var(--fc-accent);
  color: #fff;
}

.fc-btn--ghost {
  background: #eceff1;
  color: #37474f;
}

.fc-btn--wa {
  background: #25d366;
  color: #fff;
  text-decoration: none;
  display: inline-flex;
  align-items: center;
}

.fc-legal {
  margin-top: 1.5rem;
  font-size: 0.78rem;
}

/* Módulo pinturas – header y banners */
.mp-page {
  padding-top: 0;
}

```

```

}

.mp-banner {
  padding: 0.45rem 1rem;
  font-size: 0.82rem;
  font-weight: 600;
  text-align: center;
}

.mp-banner--lab {
  background: #fff3e0;
  color: #e65100;
  border-bottom: 1px solid #ffcc80;
}

.mp-banner--caja {
  background: #e8f5e9;
  color: #1b5e20;
  border-bottom: 1px solid #a5d6a7;
}

.mp-top {
  background: #fff;
  border-bottom: 1px solid #eceff1;
  padding: 0.65rem 1rem;
}

.mp-top-inner {
  max-width: 1100px;
  margin: 0 auto;
  display: flex;
  align-items: center;
  justify-content: space-between;
}

.mp-brand {
  display: flex;
  align-items: center;
  gap: 0.65rem;
  color: #263238;
}

.mp-brand strong {
  display: block;
  font-size: 0.95rem;
}

.mp-brand small {
  color: #78909c;
  font-size: 0.75rem;
}

.mp-wa {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 2.25rem;
  height: 2.25rem;
  border-radius: 50%;
  background: #25d366;
  color: #fff;
  text-decoration: none;
}

.mp-lab-page {
  background: #f5f7f8;
  min-height: 100vh;
}

```

```

.mp-lab-main {
    max-width: 640px;
    margin: 0 auto;
    padding: 2.5rem 1.25rem;
    text-align: center;
}

.mp-lab-main h1 {
    font-size: 1.75rem;
    font-weight: 800;
}

.mp-lab-hint {
    font-size: 0.82rem;
    color: #607d8b;
    text-align: left;
    background: #fff;
    border-radius: 10px;
    padding: 1rem;
    border: 1px solid #eceeff1;
}

```

scripts/generar_mascaras_ambientes.py

```

"""Genera máscaras PNG de muro para el visualizador Colores en tienda."""
from __future__ import annotations

import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT))

from PIL import Image, ImageDraw # noqa: E402

from services.fabrica_color_service import AMBIENTES # noqa: E402

OUT = ROOT / 'static' / 'img' / 'fabrica-color' / 'masks'
SIZE = (1400, 933)

def _scale(poly: list, w: int, h: int) -> list[tuple[int, int]]:
    return [(int(p[0] * w), int(p[1] * h)) for p in poly]

def _ellipse_box(ex: list, w: int, h: int) -> tuple[int, int, int, int]:
    cx, cy, rx, ry = ex
    x0 = int((cx - rx) * w)
    y0 = int((cy - ry) * h)
    x1 = int((cx + rx) * w)
    y1 = int((cy + ry) * h)
    return x0, y0, x1, y1

def build_mask(amb: dict) -> Image.Image:
    w, h = SIZE
    img = Image.new('L', (w, h), 0)
    draw = ImageDraw.Draw(img)
    polys = amb.get('wall_polygons') or []
    if not polys and amb.get('wall_polygon'):
        polys = [amb['wall_polygon']]
    for poly in polys:
        draw.polygon(_scale(poly, w, h), fill=255)

```

```

        for ex in amb.get('wall_exclusions') or []:
            draw.ellipse(_ellipse_box(ex, w, h), fill=0)
        # Sin blur: el difuminado filtraba alpha al mueble y teñía toda la foto.
        return img

def main() -> None:
    OUT.mkdir(parents=True, exist_ok=True)
    for amb in AMBIENTES:
        aid = amb['id']
        mask = build_mask(amb)
        path = OUT / f'{aid}.png'
        mask.save(path, optimize=True)
        print(f'OK {aid} -> {path} ({path.stat().st_size} bytes)')

if __name__ == '__main__':
    main()

```

tests/test_modulo_pinturas.py

```

"""Módulo cliente Fábrica de Color – lab preview + sesión caja."""
import pytest

from services import fabrica_color_service as fc
from services import modulo_pinturas_session_service as ms

@pytest.mark.smoke
def test_modulo_pinturas_lab_404_sin_preview(app_client):
    r = app_client.get('/modulos/pinturas/lab')
    assert r.status_code == 404

@pytest.mark.smoke
def test_modulo_pinturas_lab_ok_con_preview(app_client, monkeypatch):
    monkeypatch.setenv('VITRINA_FABRICA_COLOR_PREVIEW', '1')
    r = app_client.get('/modulos/pinturas/lab')
    assert r.status_code == 200
    assert b'Preview lab' in r.data
    assert b'lab/iniciar' in r.data

@pytest.mark.smoke
def test_modulo_pinturas_wizard_lab(app_client, monkeypatch):
    monkeypatch.setenv('VITRINA_FABRICA_COLOR_PREVIEW', '1')
    r = app_client.get('/modulos/pinturas/lab/iniciar')
    assert r.status_code == 200
    assert b'Colores en tienda' in r.data
    assert b'fabrica-color.js' in r.data

@pytest.mark.smoke
def test_modulo_pinturas_cotizar_lab(app_client, monkeypatch):
    monkeypatch.setenv('VITRINA_FABRICA_COLOR_PREVIEW', '1')
    from services import pintura_stock_palette_service as pss

    cols = pss.paleta_desde_stock(uso='interior', solo_con_stock=True)
    color_id = cols[0]['id'] if cols else 'p1'
    r = app_client.post(
        '/api/modulos/pinturas/lab/cotizar',
        json={
            'ambiente_id': 'living',
            'color_id': color_id,

```

```

        'brillo_id': 'mate',
        'm2': 15,
        'calidad_id': 'standard',
    },
)
assert r.status_code == 200
data = r.get_json()
assert data.get('ok') is True
assert data.get('cantidad', {}).get('galones_sugeridos', 0) >= 1
assert 'totales' in data
assert data['totales'].get('total_proyecto_fmt', '').startswith('$')
if data.get('tinte', {}).get('modo') == 'stock_erp':
    assert data.get('producto')

@pytest.mark.smoke
def test_tienda_sin_link_fabrica(app_client):
    r = app_client.get('/tienda/ferreteria-santo-domingo')
    assert r.status_code == 200
    assert b'fabrica-de-color' not in r.data
    assert b'tienda-nav-fabrica' not in r.data

@pytest.mark.smoke
def test_legacy_fabrica_vitrina_404(app_client):
    r = app_client.get('/tienda/ferreteria-santo-domingo/fabrica-de-color')
    assert r.status_code == 404

def test_caja_habilitar_modulo_pinturas(app_client):
    r = app_client.post('/api/caja/modulo-pinturas/habilitar')
    assert r.status_code == 200
    data = r.get_json()
    assert data.get('ok') is True
    assert data.get('token')
    assert data.get('path', '').startswith('/modulos/pinturas/')

def test_caja_token_abre_wizard(app_client):
    r = app_client.post('/api/caja/modulo-pinturas/habilitar')
    assert r.status_code == 200
    token = r.get_json().get('token')
    rw = app_client.get(f'/modulos/pinturas/{token}')
    assert rw.status_code == 200
    assert b'Sesi' in rw.data and b'mostrador' in rw.data
    assert b'fclizTip' in rw.data
    assert b'"modo_caja": true' in rw.data

def test_fabrica_calcular_cantidad():
    c = fc.calcular_cantidad(m2=35, manos=2)
    assert c['m2'] == 35
    assert c['galones_sugeridos'] >= 1

def test_cartilla_sd_carga():
    from services import pintura_stock_palette_service as pss

    meta = pss.resumen_stock()
    assert meta.get('modo') == 'stock_erp'
    # local QA puede tener 0 o más según stock tienda
    assert 'total_interior' in meta

def test_paleta_stock_serializa(app_ctx):
    from services import pintura_stock_palette_service as pss

    cols = pss.paleta_desde_stock(uso='interior', solo_con_stock=True)

```



```

for c in cols[:3]:
    assert c.get('id', '').startswith('p')
    assert c.get('producto_id')
    assert c.get('hex')

def test_liz_tip_reglas():
    out = fc.liz_tip_wizard(
        step=2,
        ambiente_id='bano',
        color_id='b-001',
        brillo_id='mate',
        m2=12,
    )
    assert out.get('ok') is True
    assert out.get('tip')
    assert out.get('fuente') in ('reglas', 'ollama')

@pytest.mark.smoke
def test_modulo_pinturas_liz_tip_api(app_client, monkeypatch):
    monkeypatch.setenv('VITRINA_FABRICA_COLOR_PREVIEW', '1')
    r = app_client.post(
        '/api/modulos/pinturas/lab/liz-tip',
        json={'step': 2, 'ambiente_id': 'living', 'color_id': 'b-001', 'brillo_id': 'mate', 'm2': 15},
    )
    assert r.status_code == 200
    data = r.get_json()
    assert data.get('ok') is True
    assert data.get('tip')

def test_sesion_preview_flag(monkeypatch):
    monkeypatch.setenv('VITRINA_FABRICA_COLOR_PREVIEW', '1')
    assert ms.preview_habilitado() is True
    assert ms.validar_acceso('lab') is not None
    monkeypatch.setenv('VITRINA_FABRICA_COLOR_PREVIEW', '0')
    assert ms.preview_habilitado() is False

```